University of Chinese Academy of Sciences

硕士学位论文

基于 LoongArch64 指令集的 Java 虚拟机的移植与优化

作者姓名：黄　佳

指导教师：章隆兵　副研究员

中国科学院计算技术研究所

学位类别：工学硕士

学科专业：计算机系统结构

培养单位：中国科学院计算技术研究所

2021 年 6 月

Migration and Optimization of Java Virtual Machine Based on LoongArch64 Instruction Set

A thesis submitted to

University of Chinese Academy of Sciences

in partial fulfillment of the requirement

for the degree of

Master of Science

in Engineering

By

Jia Huang

Supervisor： Professor Longbing Zhang

Institute of Computing Technology, Chinese Academy of Sciences

June 2021

中国科学院大学
研究生学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：

日　　期：

中国科学院大学
学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以按照学术研究公开原则和保护知识产权的原则公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：　　导师签名：

日　　期：　　日　　期：

摘 要

LoongArch 指令集是自主指令系统架构，具有先进性、兼容性和扩展性的特点。Java 语言是最受欢迎和应用最广泛的编程语言之一。Java 虚拟机是 Java 语言实现平台无关性、安全性和许多高级语言特性的基础，是 Java 技术体系的核心。将 Java 虚拟机移植到 LoongArch 指令集架构有助于推动龙芯自主生态的建立。

本文完成的工作和主要贡献如下：

第一，完成龙芯 LoongArch 64 位 Java 虚拟机的移植。以 OpenJDK8 为基础，将 Java 虚拟机中和平台相关的部分，如汇编器、解释器和编译器等移植到 LoongArch 指令集架构处理器上。

第二，完成龙芯 LoongArch 64 位 Java 虚拟机的优化。主要包括：1）通过优化立即数加载函数和使用移位运算指令和双寄存器访存指令优化指令分派，提高了解释器的翻译性能，使得 Java 虚拟机的启动性能平均提高了 8.94%；2）通过使用相对 PC 运算指令和调整 CodeCache 虚拟内存映射空间，优化了 Java 虚拟机中的跳转机制，使得优化后的 Java 虚拟机将近 1/3 的字节码对应的汇编指令条数减少，平均性能提升幅度为 1.59%；3）通过使用多条 LoongArch 基础指令拼凑出条件传送指令，使得 scimark.monte_carlo 程序的性能提升 71%，分支误预测率从 2.99%下降至 0.01%。

第三，进行龙芯 LoongArch 64 位 Java 虚拟机性能评测，结果表明：LoongArch 64 位 Java 虚拟机在同主频 LoongArch 架构处理器相比于 MIPS 架构处理器的 Java 虚拟机平均性能提升 21.87%，性能最高提升 42.48%。运行在 2.5GHz 主频的 LoongArch 架构处理器的 Java 虚拟机比运行在 1.8GHz 主频 MIPS 架构处理器的 Java 虚拟机平均性能提升 64.70%，性能最高提升 97.06%。

最后，基于优化和性能分析数据，提出了 LoongArch 指令集的改进方向，包括增加条件传送指令和加法进位指令。本文的研究成果已经集成到龙芯 OpenJDK8 版本库中，并得到了实际应用。

关键词： LoongArch 指令集，Java 虚拟机，OpenJDK8，优化机制

Abstract

LoongArch Instruction Set is an autonomous instruction system architecture, which has the characteristics of advanced, compatibility and expansibility. Java is one of the most popular and widely used programming languages. Java virtual machine is the basis of platform independence, security and many high-level language features of Java language. It is also the core of Java technology system. Porting Java virtual machine to LoongArch Instruction Set Architecture helps to promote the establishment of Loongson's independent ecosystem.

The contributions of this thesis are as follows:

Firstly, complete the porting of LoongArch 64 bit Java virtual machine. Based on OpenJDK8, platform-dependant parts of Java virtual machine, such as assembler, interpreter and compiler, are migrated to LoongArch Instruction Set Architecture architecture processor.

Secondly, complete the optimization of LoongArch 64 bit Java virtual machine. The main contents are as follows: 1) By optimizing the immediate loading function and using shift operation instruction and double register access instruction, the interpreter translation performance is improved, and the start-up performance of Java virtual machine is improved by 8.94%; 2) By using relative PC operation instructions and adjusting the codecache virtual memory mapping space, the jump mechanism in Java virtual machine is optimized, which makes the number of assembly instruction bars corresponding to nearly 1/3 byte codes of the optimized Java virtual machine reduced, and the average performance improved by 1.59%; 3) By using multiple LoongArch base instructions to pieced together conditional transfer instructions, the performance of scimark.monte_carlo was improved 71%, and the branch-miss rate decreased from 2.99% to 0.01%.

Thirdly, the performance evaluation of LoongArch 64 bit Java virtual machine. The average performance of the LoongArch Java virtual machine is 21.87% higher

than that of MIPS, and the performance is up to 42.48%. In the default dominant frequencies , the average performance of the Java virtual machine of LoongArch architecture is 64.70% higher than that of 1.8GHz MIPS architecture, and the performance is up to 97.06%.

Finally, based on the optimization and performance analysis data, the improvement direction of LoongArch instruction set is proposed, including adding conditional transfer instruction and adding carry instruction. The research findings of this paper have been integrated into Loongson OpenJDK8 repository, and have been applied in practice.

Key Words: LoongArch instruction set, Java Virtual Machine, OpenJDK8, Optimization Mechanism

目 录

第 1 章 绪论 1

1.1 龙芯 LoongArch 指令集架构简介 1
1.2 Java 虚拟机简介 3
1.3 龙芯 LoongArch 64 位 Java 虚拟机移植的必要性 4
1.4 相关工作介绍 5
1.4.1 HotSpot 虚拟机相关工作 5
1.4.2 龙芯平台 HotSpot 虚拟机相关工作 6
1.5 本文研究内容与研究工作 6
1.6 本文的组织结构 7

第 2 章 龙芯 LoongArch 64 位 Java 虚拟机移植 9

2.1 HotSpot 结构 9
2.2 汇编器移植 11
2.2.1 指令操作码编码实现 11
2.2.2 寄存器接口实现 11
2.2.3 定义指令编码格式 13
2.2.4 指令发射 13
2.3 解释器移植 14
2.3.1 初始化模板及生成机器指令 15
2.3.2 创建正常派遣表 17
2.3.3 指令分派 17
2.4 编译器移植 19
2.4.1 C2 编译器架构 19
2.4.2 体系结构描述文件 20
2.5 本章小结 21

第 3 章 龙芯 LoongArch 64 位 Java 虚拟机优化 23

3.1 优化解释器翻译性能 23

3.1.1 背景介绍.....23
3.1.2 立即数加载优化.....26
3.1.3 使用移位运算指令和双寄存器访存指令优化指令分派.....26
3.1.4 小结.....27
3.2 跳转机制的优化方法.....28
3.2.1 跳转机制.....28
3.2.1.1 LoongArch 跳转指令介绍.....28
3.2.1.2 Label 类.....29
3.2.1.3 原始跳转机制实现.....30
3.2.2 使用相对 PC 运算指令实现跳转机制.....31
3.2.3 调整 CodeCache 虚拟内存映射空间.....32
3.2.4 跳转机制优化实现.....37
3.2.5 小结.....39
3.3 条件传送机制的优化方法.....39
3.3.1 背景介绍.....39
3.3.2 MIPS 条件传送指令介绍.....42
3.3.3 条件传送机制优化实现.....44
3.3.3.1 原始条件传送机制实现.....44
3.3.3.2 条件传送机制优化实现.....45
3.3.4 小结.....48

第 4 章 龙芯 LoongArch64 位 Java 虚拟机性能评测.....51

4.1 测试平台和工具介绍.....51
4.2 解释器翻译性能优化效果.....52
4.2.1 静态数据对比.....52
4.2.2 启动性能.....54
4.3 跳转机制优化性能评测.....55
4.4 条件传送机制优化性能评测.....57
4.4.1 精简程序性能.....57
4.4.2 scimark.monte_carlo 程序性能.....58
4.5 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比与分析.....59

第 5 章　总结与展望 63

5.1　本文工作总结 63
5.2　龙芯 LoongArch64 位指令集的改进方向.....64
5.3　下一步工作和研究方向 64

参考文献 67

致 谢 71

作者简历及攻读学位期间发表的学术论文与研究成果.....73

图目录

图 1.1 LoongArch 指令集架构组成部分 2
图 1.2 LoongArch 架构典型指令编码格式 3
图 1.3 Java 程序执行流程 4
图 2.1 HotSpot 结构框图 9
图 2.2 定义 7 位操作码指令 12
图 2.3 jirl 指令实例化和发射 14
图 2.4 HotSpot 解释器架构 15
图 2.5 字节码指令定义 16
图 2.6 iconst 字节码模板 16
图 2.7 EntryPoint 定义 17
图 2.8 dispatch_base()方法 18
图 2.9 HotSpot C2 编译器架构 20
图 2.10 指令构造实例 21
图 3.1 原始立即数加载实现 24
图 3.2 立即数加载优化实现 25
图 3.3 长跳转指令拼接的 64 位立即数 30
图 3.4 ReservedSpace::initialize 代码片段 34
图 3.5 Java 线程运行时虚拟内存映射 35
图 3.6 LoongArch 的 scimark.monte_carlo 程序 perf 性能采集结果 41
图 3.7 MIPS 的 scimark.monte_carlo 程序 perf 性能采集结果 41
图 4.1 优化前后 LoongArch 与 MIPS 字节码对应指令条数比较 53
图 4.2 条件传送优化前后精简程序运行时间对比 57
图 4.3 条件传送机制精简测试 58
图 4.4 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比 61

表目录

表 2.1 LoongArch 浮点寄存器使用约定 12
表 2.2 LoongArch 整数寄存器使用约定 13
表 2.3 2R 类型指令编码 14
表 2.4 2RI8 类型指令编码 14
表 2.5 派遣表结构 18
表 3.1 解释器翻译性能对比 24
表 3.2 LoongArch 与 MIPS 字节码对应指令条数比较 24
表 3.3 字节码 istore_0 对应汇编指令 25
表 3.4 移位运算指令 26
表 3.5 双寄存器访存指令 27
表 3.6 dispatch_base()方法优化前后对比 27
表 3.7 字节码 istore_0 优化前后对比 28
表 3.8 LoongArch 跳转指令 29
表 3.9 LoongArch Label 的 bound 状态和 unbound 状态 30
表 3.10 原始跳转机制实现 31
表 3.11 pcaddu12i 和 pcaddu18i 指令 31
表 3.12 使用相对 PC 运算指令实现跳转机制 32
表 3.13 Java 线程运行时虚拟内存映射（新） 36
表 3.14 跳转机制优化实现 37
表 3.15 跳转机制实现对比 37
表 3.16 原始跳转机制 patch 情况分类 38
表 3.17 跳转优化机制 patch 情况分类 38
表 3.18 scimark.monte_carlo 测试结果 39
表 3.19 LoongArch 部分汇编指令 42
表 3.20 MIPS 部分汇编指令 42
表 3.21 分支指令和条件传送指令 43
表 3.22 fsel 指令 43
表 3.23 MIPS 条件传送指令 44

表 3.24 原始条件传送机制.....45
表 3.25 maskeqz 和 masknez 指令.....46
表 3.26 整数条件整数传送优化.....46
表 3.27 movgr2fr 和 movfr2gr 指令.....47
表 3.28 浮点条件整数传送优化.....47
表 3.29 fcmp.ceq 指令.....48
表 3.30 整数条件浮点传送优化.....48
表 3.31 条件传送机制.....49
表 4.1 测试平台主要配置.....51
表 4.2 优化前后 LoongArch 解释器翻译性能对比.....53
表 4.3 LoongArch 优化后与 MIPS 解释器翻译性能对比.....53
表 4.4 LoongArch Java 虚拟机启动性能.....54
表 4.5 优化前、后解释器翻译性能对比.....55
表 4.6 字节码对应汇编指令条数减少百分比.....55
表 4.7 跳转机制优化前、后 SPECjvm2008 测试数据对比表.....56
表 4.8 条件传送优化前后精简程序分支误预测率对比.....58
表 4.9 条件传送机制优化前后 scimark.monte_carlo 程序性能比较.....59
表 4.10 scimark.monte_carlo 程序 MIPS 与 LoongArch 性能比较.....59
表 4.11 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比.....60

第1章 绪论

国产 CPU 龙芯处理器采用 MIPS 指令集架构。20 世纪 80 年代早期，MIPS 指令集架构从斯坦福大学诞生，是一种采取精简指令集（Reduced Instruction Set Computing,简称 RISC）的处理器架构。时至今日，MIPS 的部分设计有些过时，不能完全适合现代的操作系统和 CPU 设计，例如存在跳转指令偏移范围小、只有 3 种指令格式、仍然保留过时的延迟槽技术等问题。在这样的背景下，龙芯处理器研究团队（简称龙芯团队）充分考虑先进与兼容需求，构建了自主指令集架构 LoongArch。

Java 语言[28]是一种面向对象的高级程序设计语言，可用于编写跨平台的应用程序，目前已广泛应用于网络服务器、桌面系统和移动终端等领域，并在全世界拥有最大的开发群体。Java 虚拟机是 Java 技术体系的核心，负责对 Java 程序进行模拟执行，是影响系统性能的关键因素。Java 虚拟机是 Java 实现平台无关性、安全性和许多高级语言特性的基础，同时也决定了 Java 的性能。

本文基于龙芯平台，进行了 LoongArch 指令集架构[2]上 Java 虚拟机的移植和优化。本文工作有助于推动龙芯自主生态的建立。

1.1 龙芯 LoongArch 指令集架构简介

LoongArch 指令集是龙芯团队充分考虑兼容需求设计的自主指令系统。LoongArch 指令集具有先进性、兼容性和扩展性三个特点[1]。先进性是指积极吸收近年来指令集发展的先进技术成果，适度摒弃一些过时的技术特征；兼容性是指融合 X86、MIPS、ARM 指令系统的主要特点，高效支持二进制翻译；扩展性是指 LoongArch 指令槽留有空余，有利于 LoongArch 指令集今后的持续迭代演进。

LoongArch 指令集架构具有 RISC 指令架构的典型特征。它的指令长度固定且编码格式规整，绝大多数指令只有两个源操作数和一个目的操作数。它采用 load/store 架构，即仅有 load/store 访存指令可以访问内存，其他指令的操作对象均是处理器核内部的寄存器或指令码中的立即数。

LoongArch 指令集架构采用基础部分（Loongson Base）加扩展部分的组织形式，如图 1.1 所示。其中扩展部分包括：二进制翻译扩展（Loongson Binary

Translation,简称 LBT）、虚拟化扩展（Loongson Virtualization,简称 LVZ）、向量扩展（Loongson SIMD Extension,简称 LSX）和高级向量扩展（Loongson Advanced SIMD Extension,简称 LASX）。LoongArch 总共具有 2565 条原生指令，其中包括基础指令 337 条、虚拟机扩展指令 10 条、二进制翻译扩展指令 176 条、128 位向量扩展指令 1024 条、高级向量扩展指令 1018 条。相对于 MIPS 指令集架构，LoongArch 摒弃了部分不适合现代 CPU 的指令，又做了大量改进和扩展。例如取消转移指令延迟槽；满足部分指令长立即数要求，单条指令支持的立即数扩展到最大 26 位；新增将 PC 作为源操作数的运算指令。这些改进和扩展都有效减少了编译结果的目标指令条数和访存次数，提高了效能。

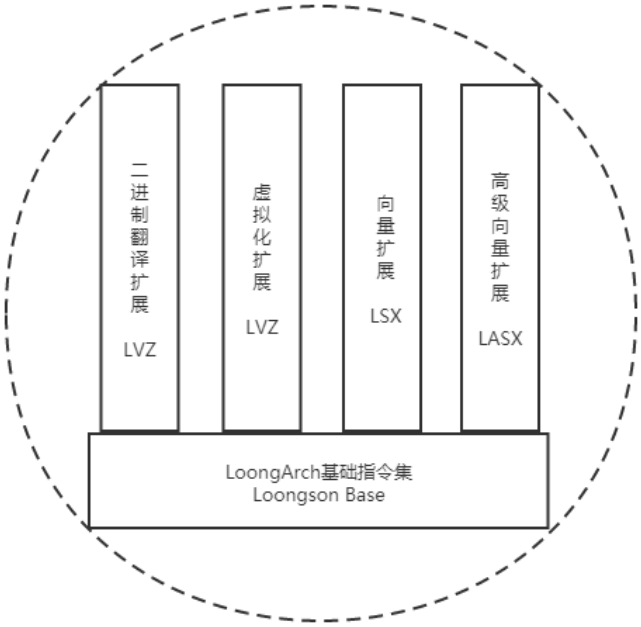

图 1.1 LoongArch 指令集架构组成部分

Figure 1.1 components of LoongArch Instruction Set Architecture

相较于 MIPS 只有 3 种指令格式，LoongArch 可用的指令格式多达 10 种。具体来说，包含 9 种典型的指令编码格式，即 3 种不含立即数的编码格式 2R、3R、4R，以及 6 种含立即数的编码格式 2RI8 、2RI12、2RI14、2RI16、2RI21、2RI26。图 1.2 列举了这 9 种典型编码格式的具体定义。还存在少数指

令，其指令编码域并不完全等同于这 9 种典型指令编码格式，而是在其基础上略有变化。

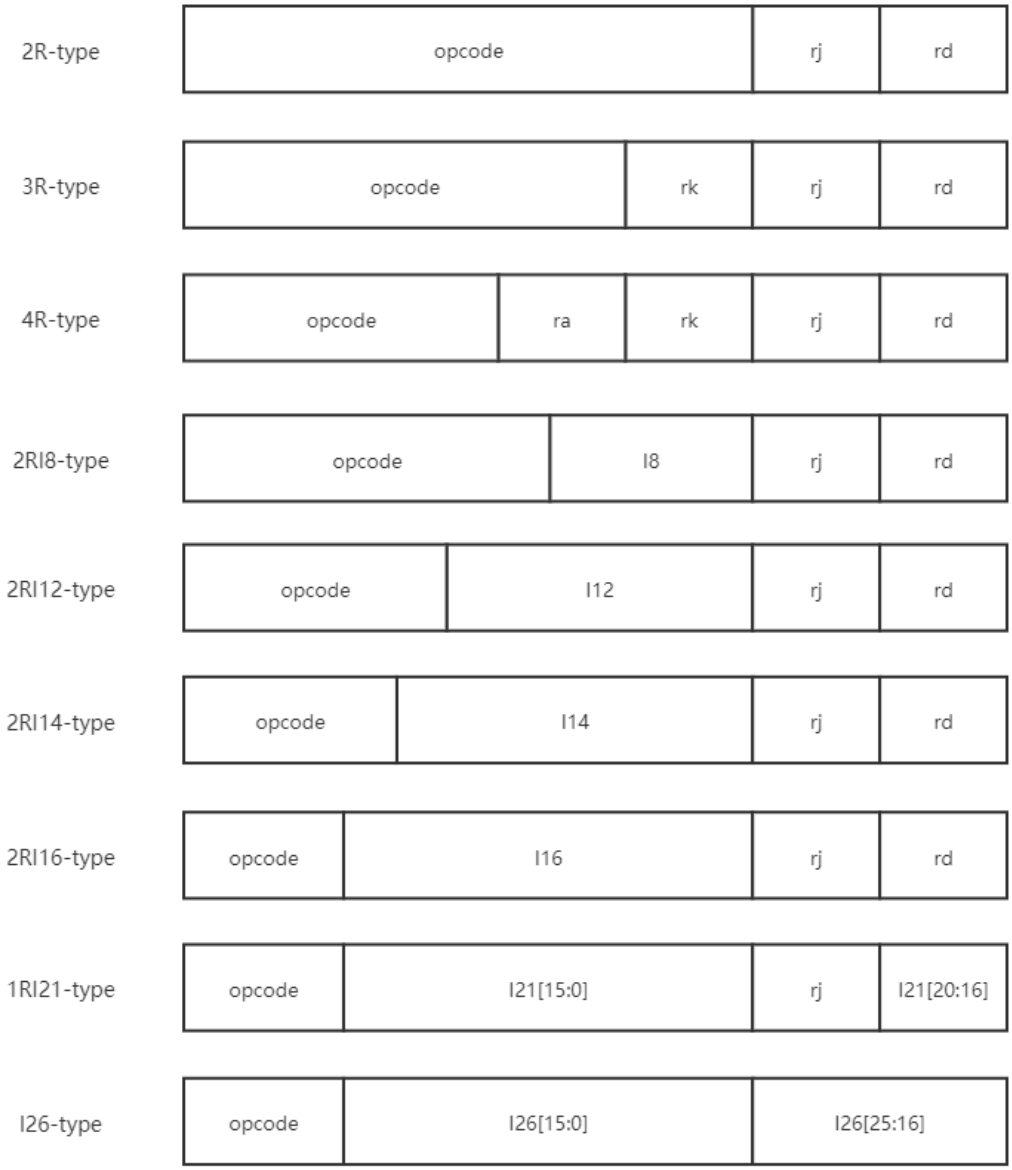

图 1.2 LoongArch 架构典型指令编码格式

Figure 1.2 Typical instruction encoding formats of LoongArch Architecture

1.2 Java 虚拟机简介

Java 技术的核心是 Java 虚拟机，所有的 Java 程序都在虚拟机上运行。Java 程序的执行依赖于编译环境和运行环境，执行流程见图 1.3。Javac 编译器将

Java 源代码翻译成 Java 字节码。Java 虚拟机加载、执行通过本地或者网络传递的 Java 字节码文件，对其屏蔽底层操作系统和硬件的差异性。正是因为如此，Java 程序才能达到“write once, run anywhere”的功能特性。Java 虚拟机实例负责运行一个 Java 程序。当启动一个 Java 程序时，一个虚拟机实例就诞生了。当程序结束，这个虚拟机实例也就消亡。

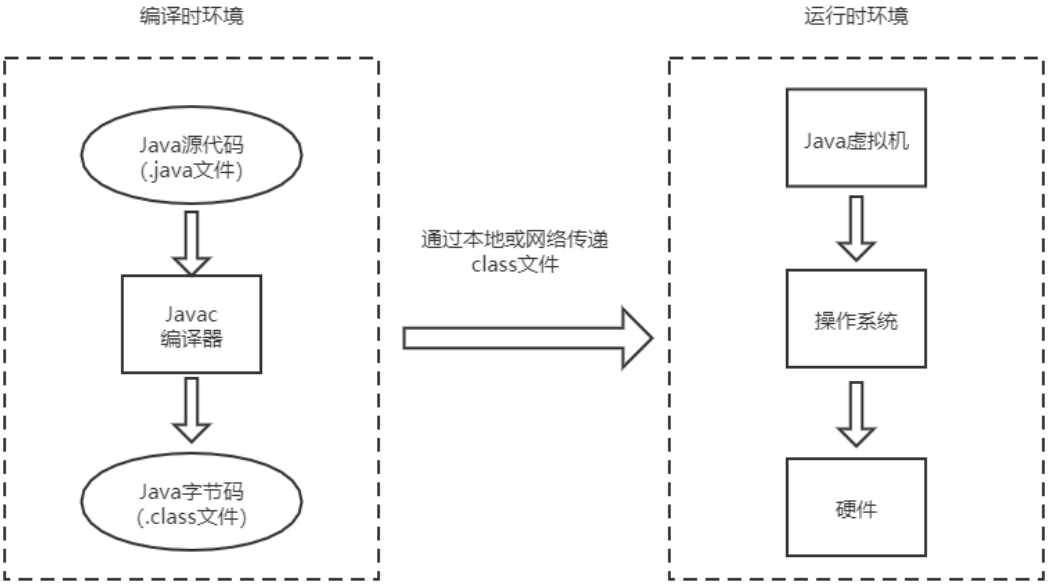

图 1.3 Java 程序执行流程

Figure 1.3 Java program execution process

1.3 龙芯 LoongArch 64 位 Java 虚拟机移植的必要性

世界著名的编程语言排行榜 TIOBE 在 2021 年 4 月的统计数据[14]表明，Java 的流行度位居第二，人气值高达 11.23%。Java 编程语言被广泛的应用于各种生产环境中，其应用范围包括嵌入式系统、移动终端、企业服务器、大型机等各种场合。Java 在全球各地有超过 900 万的开发人员，使用 Java 的设备多达几十亿台，其中包括 11 亿台个人计算机、30 亿部移动电话及其他手持设备、数量众多的光盘播放器、TV 设备和其他设备[15]。

Java 语言能够获得如此广泛的认可，除了它是一门结构严谨、面向对象的编程语言之外，它还有许多不可忽视的优点和特性：它摆脱了硬件平台的束缚，实现了“write once, run anywhere”的理想；它提供了一个相对安全的内存

管理和访问机制，避免了绝大部分的内存泄露和指针越界问题；它实现了热点代码检测和运行时编译及优化，这使得 Java 应用能随着运行时间的增加而获得更高的性能[16]。这些优点极大地便利了程序员，使得程序员的开发效率得到了很大的提升，同时也使得 Java 语言被广泛应用。

Java 虚拟机是实现这些优点和特性的基础，是 Java 技术体系的核心。将 Java 虚拟机移植到 LoongArch 指令集架构，意味着大量的 Java 应用程序均可运行在龙芯 LoongArch 平台，有助于推动龙芯自主生态的建立。

1.4 相关工作介绍

HotSpot 虚拟机是 OracleJDK 和 OpenJDK 中的默认的 Java 虚拟机，也是目前使用范围最广的 Java 虚拟机。本节首先对 HotSpot 虚拟机进行了简要的介绍，接着介绍了龙芯平台 HotSpot 虚拟机的相关工作。

1.4.1 HotSpot 虚拟机相关工作

HotSpot[39]虚拟机最初由一家名为“Longview Techno-logies”的公司开发，由于 HotSpot 虚拟机优异的性能表现，Sun 公司于 1997 年收购了该公司。HotSpot 虚拟机成为 JDK1.3 及之后所有 JDK 版本的默认虚拟机，占据绝对的市场地位。

HotSpot 虚拟机经过 20 多年的发展，引入了很多重要的新特性[17]。JDK5 时引入了 CDS（Class-Data Sharing）。CDS 允许将一组类预处理到一个共享的归档文件中，后续 HotSpot 虚拟机运行时将归档文件映射到内存中，以提升 Java 应用的启动速度。JDK6 时引入了分层编译。在分层编译的工作模式出现前，HotSpot 虚拟机通常采用解释器与其中一个编译器直接搭配的方式工作。为了在程序启动速度与运行效率之间达到最佳平衡，HotSpot 虚拟机在编译子系统中加入了分层编译功能。JDK8 中移除了 HotSpot 虚拟机中的永久代，对 Lambda 表达式提供了支持。JDK 9 最大的变化之一是引入了模块系统。JDK9 将 JDK 分成一组模块，可以在构建时、编译时和运行时进行组合。模块化可以减少内存开销，减少各种类库、大型应用的开发和维护。JDK9 另外一个重要的变化是 G1GC 取代了 Parallel GC 成为了 HotSpot 虚拟机中默认的垃圾回收机制。因为对于大多数用户而言，G1GC 的低暂停时间特性比 Parallel GC 的高吞

吐量特性更重要。JDK11 引入了 JFR（Java Flight Record）。JFR 是 Java 平台性能分析和事件采集的框架。它能够采集到 low-level 的性能信息，同时对 Java 程序的性能影响可以忽略不计。JDK12 引入了 Microbenchmark Suite 功能。该功能为 JDK 源代码添加了一套基于 JMH（Java Microbenchmark Harness）的微基准测试，使开发人员可以轻松地运行现有的微基准测试并创建新的微基准测试。

1.4.2 龙芯平台 HotSpot 虚拟机相关工作

龙芯处理器 2001 年从中科院计算所诞生，指令集架构为并不主流的 MIPS 架构。Java 虚拟机的设计不依赖某一特定平台，但是社区中主要还是针对 X86、ARM 等主流架构进行实现，在 MIPS 架构缺少相应的实现或优化。为了使龙芯处理器能够支持 Java 运行环境，2006 年刘超[4]将 JDK5 移植到龙芯 2 号平台上，填补了龙芯平台在 Java 虚拟机上的空白。2007 年陈杰[5]在刘超工作的基础上，完善龙芯 2 号平台上的 JDK5，并通过改写 Cache 操作方式和处理非对齐地址的访问，提高了龙芯平台 Java 虚拟机的性能和稳定性。与此同时，伍鸣[6]深入研究了 Java 虚拟机中的垃圾回收机制，对龙芯平台 Java 虚拟机的垃圾回收机制进行了优化。2012 年杨勇强[7]将 JDK6 移植到龙芯 3 号平台，蔡嵩松[8]通过软硬件协同设计优化龙芯 3 号平台的 Java 虚拟机。2013 年武慧[9]以 JDK6 为基础，实现了基于龙芯 MIPS64 平台的 Java 虚拟机系统。这也是龙芯平台首次支持 64 位 Java 虚拟机。2015 年王一光[10]使用 JNI 调用机制和 Intrinics 机制优化热点方法，提升 Java 虚拟机在龙芯 MIPS64 平台的性能。2016 年敖琪[11]在 Java 虚拟机层面进行了容错技术的研究，傅杰[12]从动态编译和解释执行两方面对 Java 虚拟机性能优化技术进行了研究。2018 年陈昊轩[13]以 JDK8 为基础，在龙芯 MIPS64 平台实现了分层编译。

经过十来年的技术积累，龙芯团队对于 Java 虚拟机中的解释器、JIT 编译器、分层编译、垃圾回收机制等各项技术已经有了深刻的认识和实践，研究成果均集成到龙芯 JDK 版本库，投入实际生产。

1.5 本文研究内容与研究工作

本文的研究内容与研究工作有以下三个方面：

1. 龙芯 LoongArch 64 位 Java 虚拟机移植

LoongArch 指令集是龙芯团队设计的自主指令系统，Java 虚拟机是 Java 程序能够跨平台运行的基础。本文基于龙芯平台，完成了 LoongArch 指令集架构上 Java 虚拟机的移植，有助于推动龙芯自主生态的建立。

2. 龙芯 LoongArch 64 位 Java 虚拟机优化

利用 LoongArch 指令集的特点，本文对龙芯 LoongArch 64 位 Java 虚拟机进行了优化，提高了虚拟机性能。主要优化措施如下：1）利用 LoongArch 指令集中的双寄存器访存指令和移位运算指令优化了指令分派过程；2）利用 LoongArch 指令集中的相对 PC 运算指令减少了跳转指令的条数；3）LoongArch 指令集中缺少条件传送指令，利用 LoongArch 基础指令集中的指令拼凑出条件传送指令，使得不需要分支指令即可实现条件传送。

3. 龙芯 LoongArch 64 位 Java 虚拟机性能评测

采用国际上标准性能评估机构提供的测试工具 SPECjvm2008，对龙芯 LoongArch 64 位 Java 虚拟机进行了性能评测和性能分析，并提出了进一步的改进方向。

1.6 本文的组织结构

本文一共分为 5 章，内容分别如下。

第一章为绪论部分。首先对龙芯 LoongArch 指令集和 Java 虚拟机进行了简要的介绍，接着论证了龙芯 LoongArch 64 位 Java 虚拟机移植的必要性，然后介绍了 HotSpot 虚拟机的相关工作，最后简要地介绍了本文的研究内容与研究工作。

第二章为龙芯 LoongArch 64 位 Java 虚拟机移植部分。主要从寄存器移植、汇编器和宏汇编器移植、模板解释器移植和编译器移植这几部分来介绍。

第三章为龙芯 LoongArch 64 位 Java 虚拟机优化部分。根据性能测试结果和 LoongArch 指令集的特点，提出了三种优化方法，包括：通过优化立即数加载函数和使用移位运算指令和双寄存器访存指令优化解释器翻译性能、使用相对 PC 运算指令和调整 CodeCache 虚拟内存映射空间优化跳转机制、利用多条 LoongArch 基础指令拼凑出条件跳转指令优化条件传送机制。

第四章为龙芯 LoongArch 64 位 Java 虚拟机性能评测部分。首先简要地介绍

了测试平台和工具；对提出的 3 种优化方法进行了性能评测；最后将 LoongArch 64 位 Java 虚拟机与 MIPS 架构下的 Java 虚拟机进行了同主频和默认主频的性能对比。

第五章为总结与展望部分。首先对本文的工作进行了总结；接着在基于本文的工作和实验的前提下，提出龙芯 LoongArch 64 位指令集在 Java 虚拟机上有改进方向；最后分析了本文工作中的不足之处，进而指出了下一步工作和研究方向。

第2章 龙芯 LoongArch 64 位 Java 虚拟机移植

本章基于 OpenJDK8 在龙芯平台上移植了 LoongArch 64 位的 Java 虚拟机。OpenJDK8 包含基本类库接口 jdk 模块、处理 XML 文档编程接口 jaxp 模块、提供面向对象应用程序体系规范接口 corba 模块、处理 Web 服务接口 jaxws 模块和虚拟机实现模块 HotSpot 五个模块。其中 HotSpot 模块是 OpenJDK8 的核心模块，也是平台相关的模块。因此本章主要介绍 HotSpot 模块。

2.1 HotSpot 结构

Java 虚拟机的结构框图见图 2.1，主要由类装载子系统、运行时数据区、垃圾收集器、执行引擎和本地库接口这五个部分组成。

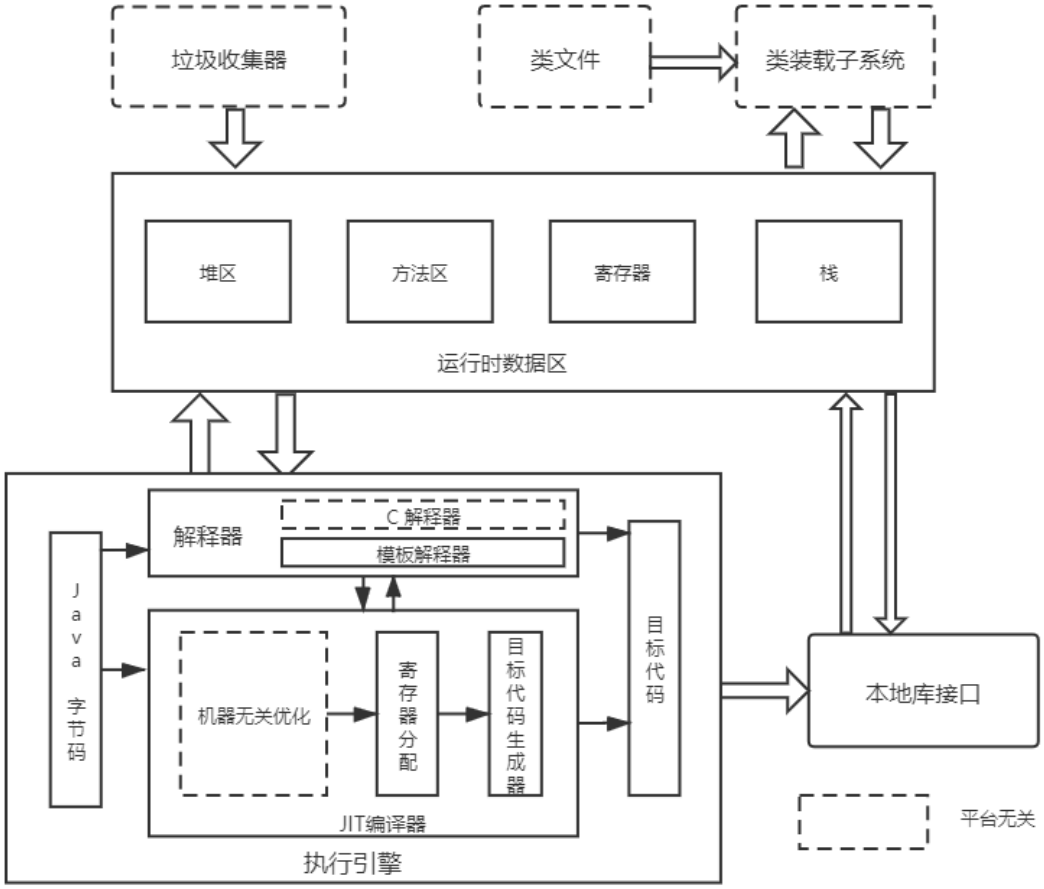

图 2.1 HotSpot 结构框图

Figure 2.1 Structure diagram of HotSpot virtual machine

类装载子系统从文件系统或者网络中加载 Class 文件载入运行时数据区。

运行时数据区包括五部分：程序寄存器、虚拟机栈、本地方法栈、堆和方法区。其中程序寄存器、虚拟机栈和本地方法栈是每个线程私有内存区域，而方法区和堆是所有线程共享内存区域。程序寄存器可以看作是当前线程所执行的字节码的行号指示器。程序执行过程中，通过改变这个计数器的值来选取下一条需要执行的字节码指令来完成下一步的操作。虚拟机栈描述的是 Java 方法执行的内存模型。整个线程执行过程中，每个方法对应着虚拟机栈中的一个栈帧，方法调用对应着一个栈帧的入栈，方法执行结束对应着一个栈帧的出栈。本地方法栈与虚拟机栈所发挥的作用是非常相似的，它们之间的区别在于虚拟机栈为虚拟机执行 Java 方法，而本地方法栈则为虚拟机执行 Native 方法。堆在虚拟机启动时创建，用于存放对象实例，是垃圾收集器管理的主要区域，因此也被称为 GC 堆。方法区用于存储已被虚拟机加载的类信息、常量、静态变量、JIT 编译器编译后的代码等数据。运行时常量池是方法区的一部分，用于存放编译器生成的各种字面量和符号引用。

垃圾收集器管理运行时数据区中的堆区，负责对象内存空间的分配，并对无用的对象进行垃圾清空和内存回收。

执行引擎是 Java 虚拟机的核心部件。执行引擎负责将 Java 字节码翻译成本地汇编代码。执行引擎包括解释器和 JIT 编译器。解释器又分为 C 解释器和模板解释器。C 解释器将字节码翻译成高级语言 C 语言进行解释执行，因此可以不做任何修改即可运行在支持 C 编译器的任何平台。模板解释器则是将字节码直接解释为本地汇编代码。C 解释器由于需要先转换为高级语言 C 语言，再转换为目标代码，因此解释执行效率更低。与之相反，模板解释器不需要经过中间过程，直接转换为汇编代码，因此执行效率更高。JIT 编译器将解释器收集到的热点方法编译成本地汇编代码后执行整个方法。字节码既可以通过解释器执行，也可以通过编译器执行。Java 程序一开始都是通过解释器执行，解释器在执行的过程中收集性能信息，统计方法和循环体执行的次数，一旦某个方法或循环体达到热点阈值，JIT 编译器则将该方法编译成本地汇编代码执行。JIT 编译执行过程中，发现编译好的本地汇编代码过于激进，如加载了新类后类型继承结果出现变化，这时也可以通过逆优化退回到解释器状态解释执行。

本地库接口是对 Java 语言中 JNI（Java Native Interface）接口的实现，它使

得 Java 虚拟机内部运行的 Java 代码能够与用其他编程语言如 C、C++编写的应用程序和库进行互操作。

HotSpot 结构中除了类装载子系统、垃圾收集器和编译器中的机器无关优化以外，其他部分都与目标平台紧密相关，因此接下来从这些方面进行移植。

2.2 汇编器移植

Java 虚拟机使用汇编器生成代码，它主要由 AbstractAssembler、MacroAssembler 和 Label 这三个类组成。AbstractAssembler 类表示纯汇编器，将机器指令发射到 CodeBuffer 中，纯汇编器在指令级别不会做任何优化。MacroAssembler 类表示宏汇编器，将频繁使用的指令序列定义为宏指令。Label 类表示控制转移指令的目标，服务于 Java 虚拟机的跳转机制。AbstractAssembler 类是汇编器的核心，接下来介绍将机器指令发射到 CodeBuffer 的实现。

2.2.1 指令操作码编码实现

将机器指令发射到 CodeBuffer 的第一步是定义 LoongArch 基础指令的操作码。在 cpu/LoongArch/vm/assembler_LoongArch.hpp 文件中，定义指令操作码。根据操作码的位数，将 LoongArch 337 条基础指令分为 6 位、8 位、10 位、12 位、14 位、17 位、22 位操作码七种方式枚举实现，由于篇幅有限，图 2.2 仅展示 7 位操作码的指令，其他位数操作码的指令同理。

2.2.2 寄存器接口实现

寄存器和指令的发射过程也是密切相关的。Java 虚拟机通过寄存器抽象接口调用目标平台的寄存器。

表 2.2 和表 2.1 分别给出了 LoongArch 整数寄存器和浮点寄存器使用规定。LoongArch 指令集架构有 32 个通用寄存器，每一个通用寄存器的位宽均为 64 位。LoongArch 指令集架构有 32 个浮点寄存器，仅当只实现操作单精度浮点数和字整数的浮点指令时，浮点寄存器的位宽为 32 位，通常情况下，浮点寄存器的位宽为 64 位。

HotSpot 虚拟机中架构层和寄存器相关的文件主要有 3 个，分别是 register_definitions_LoongArch.cpp、register_LoongArch.hpp 和

register_LoongArch.cpp。register_definitions_LoongArch.cpp 文件对通用寄存器和浮点寄存器进行了声明和定义，register_LoongArch.cpp 文件对通用寄存器、浮点寄存器和条件标志寄存器进行了别名的声明。register_LoongArch.hpp 文件对通用寄存器和浮点寄存器分别按照表 2.2 和表 2.1 的顺序进行了编码，同时对栈顶缓存寄存器（两个整型寄存器 FSR、SSR 和两个浮点寄存器 FSF、SSF）、字节码指针寄存器 BCP、局部变量指针寄存器 LVP 等 Java 虚拟机特殊用途寄存器进行了声明，将这些常用的寄存器进行了宏定义，固定的指向某个寄存器，便于开发。

```
// cpu/LoongArch/vm/assembler_LoongArch.hpp
// 7-bit opcode, highest 7 bits: bits[31...25]
enum ops7 {
        lu12i_w_op     = 0b0001010,
        lu32i_d_op     = 0b0001011,
        pcaddi_op      = 0b0001100,
        pcalau12i_op   = 0b0001101,
        pcaddu12i_op   =    0b0001110,
        pcaddu18i_op   =    0b0001111,

        unknow_ops7    = 0b1111111
};
```

图 2.2 定义 7 位操作码指令

Figure 2.2 Define 7-bit opcode instruction

表 2.1 LoongArch 浮点寄存器使用约定

Table 2.1 LoongArch float register usage convention

寄存器编号	名称	用法
$f0-$f7	fa0-fa7	参数寄存器
$f0-$f1	fv0-fv1	返回值
$f8-$f23	ft0-ft15	临时寄存器
$f24-$f31	fs0-fs7	子程序寄存器变量

表 2.2 LoongArch 整数寄存器使用约定

Table 2.2 LoongArch integer register usage convention

寄存器编号	名称	用法
$r0	zero	始终为 0
$r1	ra	子程序返回地址
$r2	tp	支持 TLS
$r3	sp	堆栈指针
$r4-$r11	a0-a7	参数寄存器
$r4-$r5	v0-v1	返回值
$r12-$r20	t0-t8	临时寄存器
$r21	x	预留，以后扩展 ABI
$r22	fp	帧指针
$r23-$r31	s0-s8	子程序寄存器变量

2.2.3 定义指令编码格式

对指令操作码和寄存器进行编码以后，就可以定义指令编码格式了。LoongArch 指令集架构包含 9 种典型的指令编码格式，3 种不含立即数的编码格式和 6 种含立即数的编码格式，具体介绍见 1.1 节。指令编码的实现位于 assembler_LoongArch.hpp 中，表 2.3 和表 2.4 分别展示了不含立即数 2R 类型和含立即数 2RI8 类型的指令编码，其他 2 种不含立即数和 5 种含立即数类型的指令编码都和这 2 个类似，篇幅有限，就不一一赘述了。其中 op 表示操作码编码，rj 和 rd 表示寄存器编码，imm8 表示指令中的立即数，分别将它们按照指令类型的格式左移到指定的位置，最后用或运算拼接，即可得到 Java 虚拟机中一条完整的机器指令编码。

2.2.4 指令发射

根据指令的编码格式，逐一实例化指令并发射，如图 2.3 中 jirl 指令。指令发射函数 emit_int32 的声明和定义见公共层汇编器的 AbstractAssembler 类中，它将该条指令编码写入代码缓冲区 CodeBuffer 中的末尾地址里，同时将末尾地

址往下移动 4 位。至此，一条完整的机器指令被发射到 CodeBuffer 中，即可被 Java 虚拟机执行引擎解释或编译执行。

表 2.3 2R 类型指令编码

Table 2.3 2R-type instruction encoding

```
// 2R-type
//   31                                  10 9          5 4         0
// |     opcode                            |     rj     |   rd     |
static inline int insn_RR   (int op, int rj, int rd) { return (op<<10) | (rj<<5) | rd; }
```

表 2.4 2RI8 类型指令编码

Table 2.4 2RI8-type instruction encoding

```
// 2RI8-type
//   31                        18 17        10 9          5 4         0
// |     opcode                  |       I8     |       rj    |   rd     |
static inline int insn_I8RR (int op, int imm8, int rj, int rd)
{ return (op<<18) | (low(imm8, 8)<<10) | (rj<<5) | rd; }
```

```
// jirl 指令实现
void jirl(Register rd, Register rj, int offs)
{
   assert(is_simm(offs, 16), "not a signed 26-bit int"); emit_int32(insn_I16RR(jirl_op,
   offs, (int)rj->encoding(), (int)rd->encoding()));
}
```

图 2.3 jirl 指令实例化和发射

Figure 2.3 jirl instruction instantiation and launch

2.3 解释器移植

HotSpot 虚拟机中有两个解释器实现：基于 C++的解释器和基于汇编的模板

解释器，它的总体架构图如图 2.4。其中 bytecodeInterpreter 和 templateTable 负责字节码的解释，cppInterpreterGenerator 和 templateInterpreterGenerator 负责解释器的运行时，它们共同完成解释功能。

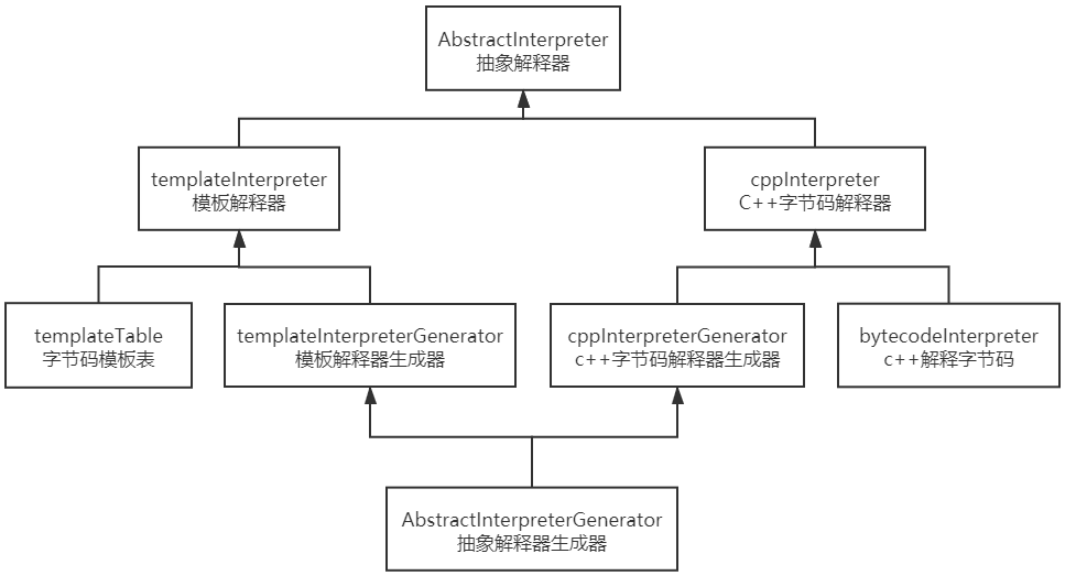

图 2.4 HotSpot 解释器架构

Figure 2.4 Architecture of the Java HotSpot interpreter

C++字节码解释器完全由高级语言 C++实现，它使用 C++语言模拟字节码的执行。基于汇编的模板解释器，它将每一条字节码指令都对应一个机器码模板。由于不需要翻译成中间运行，所以运行速度较快。HotSpot 虚拟机中默认使用模板解释器，因此接下来主要关注模板解释器。

2.3.1 初始化模板及生成机器指令

模板解释器移植的第一步是模板的初始化及机器指令的生成。平时所说的字节码 nop，其实是字节码指令的助记符，真正的字节码指令是一个数字，如字节码 nop 对应数字 0，虚拟机执行 0 这个字节码指令时，其实就是在执行 nop 字节码。在 share/vm/interpreter/bytecodes.hpp 文件中定义字节码指令，具体实现见图 2.5。

```
class Bytecodes: AllStatic {
 public:
  enum Code {
     _illegal          =          -1,

     // Java bytecodes
     _nop                          =  0, // 0x00
     _aconst_null        =  1, // 0x01
     _iconst_m1          =  2, // 0x02
     _iconst_0           =  3, // 0x03
     _iconst_1           =  4, // 0x04
     _iconst_2           =  5, // 0x05
     ......
  }
```

图 2.5 字节码指令定义

Figure 2.5 Bytecode specification

在 cpu/LoongArch/vm/templateTable_LoongArch_64.cpp 中，Java 虚拟机为每个字节码指令都创建一个模板，每个模板都关联其对应的汇编代码生成函数。以图 2.6 中 iconst 字节码为例，iconst 字节码表示将整型值 value 压入 Java 栈顶。

```
void TemplateTable::iconst(int value) {
    transition(vtos, itos);      //  验证栈顶缓存器状态
    if (value == 0) {
      __ move(FSR, R0);     //  将 0 压入栈顶寄存器 FSR
    } else {
      __ move(FSR, value);   //  将 value 值压入栈顶寄存器 FSR
    }
}
```

图 2.6 iconst 字节码模板

Figure 2.6 iconst bytecode template

在定义完所有字节码对应的模板后，模板解释器生成器中的 set_entry_points_for_all_bytes()函数会遍历所有字节码，为每个字节码生成对应

的机器指令入口，并且最终通过调用 generate_and_dispatch()函数来生成机器指令。

2.3.2 创建正常派遣表

在 2.3.1 中，为每个字节码生成了对应的机器指令入口地址。为了存储所有字节码对应机器指令入口地址，通过创建一个 EntryPoint 来记录生成的机器指令的入口地址，并将 EntryPoint 以字节码为索引，存储到正常派遣表中。EntryPoint 的定义见图 2.7，其中 btos、ztos 等描述的是当前栈顶数据的类型，总共有 10 种类型。派遣表是一个二维数组，行表示数据类型，列表示字节码指令编号，存储的元素为模板入口地址。根据不同的栈顶数据类型，机器指令的入口地址也有不同。派遣表的结构见表 2.5。

```
EntryPoint::EntryPoint(address bentry, address zentry, address
centry, address sentry, address aentry, address ientry, address
lentry, address fentry, address dentry, address ventry) {
  assert(number_of_states == 10, "check the code below");
  _entry[btos] = bentry;   // byte 型数据
  _entry[ztos] = zentry;   // byte 型数据
  _entry[ctos] = centry;   // char 型数据
  _entry[stos] = sentry;   // short 型数据
  _entry[atos] = aentry;   // 引用型数据
  _entry[itos] = ientry;   // int 型数据
  _entry[ltos] = lentry;   // long 型数据
  _entry[ftos] = fentry;   // float 型数据
  _entry[dtos] = dentry;   // double 型数据
  _entry[vtos] = ventry;   // 寄存器状态为空
}
```

图 2.7 EntryPoint 定义

Figure 2.7 Definition of Entrypoint

2.3.3 指令分派

在每一个字节码对应生成的机器指令末尾，会插入一段跳转到下一个字节码的逻辑。这样当前字节码在完成自己的功能后，就会自动取出方法体中排在它后面的字节码的指令开始执行。这个称为指令分派[30]过程。指令分派对应于 cpu/LoongArch/vm/interp_masm_LoongArch_64.cpp 中的 dispatch_base()方法，图 2.8 展示了 dispatch_base()方法中的核心代码部分。指令分派的核心在于获取下一个需要执行字节码的入口地址，该地址由 S8、table_offset、

（Rnext<<3）3 部分组成。S8 表示模板表中 itos 类型的起始地址，table_offset 表示该字节码类型的起始地址距离 itos 类型的偏移量，（Rnext<<3）表示该字节码的入口地址距离该字节码类型的起始地址的偏移量，通过 S8+table_offset+（Rnext<<3）即可获取下一个需要执行字节码的入口地址。

表 2.5 派遣表结构

Table 2.5 Dispatch Table Structure

	0	1	2	3	4	5	6	…	255
btos									
ztos									
ctos									
stos									
atos									
itos									
ltos									
ftos									
dtos									
vtos									

```
slli_d(T2, Rnext, LogBytesPerWord);
if(table_offset != 0) {
    if (is_simm12(table_offset)) {
        addi_d(T3, S8, table_offset);
    }
    else {
        li(T3, table_offset);
        add_d(T3, S8, T3);
    }
    add_d(T3, T2, T3);
    ld_d(T3, T3, 0);
}
else {
    add_d(T3, T2, S8);
    ld_d(T3, T3, 0);
}
```

图 2.8 dispatch_base()方法

Figure 2.8 dispatch_ base () method

综上，模板解释器的执行主要分为以下几步：

1. 初始化模板及生成机器指令；
2. 模板解释器生成器生成每个字节码指令的入口地址
3. 将字节码指令的入口地址存储到正常派遣表中
4. 在每个字节码对应生成的指令末尾，插入跳转到下一个字节码指令的代码。

2.4 编译器移植

HotSpot 虚拟机里内置了两个编译器，分别称为 Client Compiler[22]和 Server Compiler[23]，或者简称为 C1 编译器和 C2 编译器。C1 编译器有较快的启动速度，简单地将字节码编译成机器码。C2 编译器比 C1 编译器性能更高，提供了内联优化[37]、循环展开、DeadCode 删除[35]、分支预测[36]等优化功能。C1 编译器的移植需要涉及到大量公共层代码的修改，且 Java 虚拟机的峰值性能主要由 C2 决定，本文接下来主要关注 C2 编译器的移植过程。

2.4.1 C2 编译器架构

HotSpot C2 编译器架构[24]见图 2.9，该图展示了 C2 编译器处理一个方法的步骤。C2 编译器从一个空图开始，在解析字节码的过程中逐步添加节点。当一个节点被添加后，C2 编译器开始在本地执行一系列优化：常量折叠、标识和全局值编号[34]。之后，它通过正确地构建方法出口并执行死代码消除使 Ideal Graph 更清晰。

接下来的步骤是对 Ideal Graph 做全局优化。这些优化不是强制性的，可以被编译器跳过。在应用迭代全局值编号算法之后，ideal loop 步骤最多执行三次。ideal loop 阶段进行循环剥离、循环展开和迭代拆分等操作。条件常量传播是一种优化，它将常量传播和 if 条件结果为常数的条件结合起来。最后，再次执行迭代全局值优化和 ideal loop 阶段，即可生成优化后的 Ideal Graph。

代码生成阶段首先将 Ideal Graph 转换为特定于机器的 MachNode Graph，然后通过控制流图建立基本块、执行图形着色寄存器分配，最后经过一些窥孔优化，即可产生机器码。

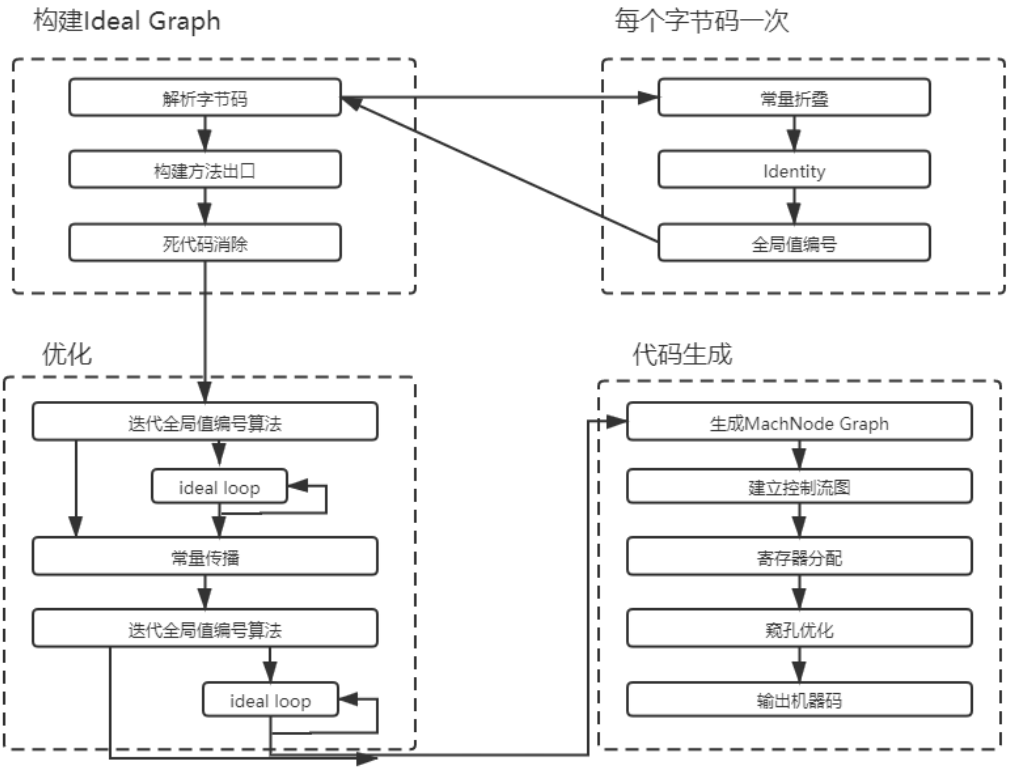

图 2.9 HotSpot C2 编译器架构

Figure 2.9 Architecture of the Java HotSpot server compiler

2.4.2 体系结构描述文件

HotSpot C2 编译器的工作流程主要分为解析字节码、消除死代码、全局优化、寄存器分配、窥孔优化和生成机器代码等步骤。其中既包括平台相关部分，也包括平台无关部分。本文关心的部分在于平台相关部分，主要涉及到部分全局优化、寄存器分配、窥孔优化和生成机器码。

实现 C2 编译器平台部分的关键在于，体系结构描述文件的编写。该文件采用体系结构描述语言（Architecture Description Language，简称 ADL）编写。这种语言用于描述处理器的体系结构，是 ADL 编译器的输入。ADL 编译器将 ADL 文件编译成代码，并将其整合到优化的编译器中，为目标体系结构生成高效且正确的代码。ADL 文件描述了三种不同类型的架构特征：目标架构的指令集和相关操作数、目标体系结构的寄存器以及寄存器分配的相关信息、目标体系结构的流水线描述。图 2.10 展示了 LoongArch 虚拟机中 ADL 文件减法指令构造实例，指令构造定义新的指令或指令序列，指定发出指令的条件，并调用

编码指令的例程。第一行，instruct subI_Reg_Reg(mRegI dst, mRegI src1, mRegI src2)定义了新的指令序列的名字 subI_Reg_Reg，并且列出了在该指令构造中要使用的所有参数。在该例中，参数是两个源寄存器，一个目的寄存器，它们存储的都是整型数据。match(Set dst (SubI src1 src2))代表匹配 Ideal Graph 中节点的模式；ins_cost 行指定生成此指令的成本花费；format 行中的内容在启用“-XX：+PrintOptoAssembly”参数时会打印出来便于程序员分析和调试；ins_encode 代码块编码指令序列并且调用相应的汇编指令例程；ins_pipe 行指定流水线的类型。

```
instruct subI_Reg_Reg(mRegI dst, mRegI src1, mRegI src2) %{
  match(Set dst (SubI src1 src2));       // 匹配模式
  ins_cost(100);              // 代价
  format %{ "sub    $dst,     $src1, $src2 #@subI_Reg_Reg" %}
  ins_encode %{
     Register  dst = $dst$$Register;         // 目的寄存器
     Register src1 = $src1$$Register;      // 源寄存器 1
     Register src2 = $src2$$Register;      // 源寄存器 2
     __ sub_w(dst, src1, src2);              // 调用汇编例程编码 sub_w 指令
  %}
  ins_pipe( ialu_regI_regI );                // 流水线类型
%}
```

图 2.10 指令构造实例

Figure 2.10 Example of instruction construction

2.5 本章小结

本章主要从汇编器移植、解释器移植和编译器移植这三部分详细介绍了龙芯 LoongArch 64 位 Java 虚拟机的移植过程。汇编器移植主要涉及到指令操作码编码实现、寄存器接口实现、定义指令编码格式和指令发射这 4 部分。模板解释器的执行先初始化字节码模板及生成机器指令；接着模板解释器生成器生成每个字节码指令的入口地址；然后将字节码指令的入口地址存储到正常派遣表中；最后在每个字节码对应生成的指令末尾，插入跳转到下一个字节码指令的代码。编译器移植的关键在于体系结构描述文件的编写，因此以 ADL 文件中减法指令构造为实例讲解了体系结构描述语言的规则。

第3章 龙芯 LoongArch 64 位 Java 虚拟机优化

将 Java 虚拟机移植到 LoongArch 指令集架构后，本章基于 LoongArch 指令集架构自身特点，对其做了一系列的优化，分别是优化解释器翻译性能、优化跳转机制和优化条件传送机制。这些优化实现对于 LoongArch 64 位 Java 虚拟机有明显的性能提升。本章对这三项优化的背景和实现进行了详细的介绍，具体的性能优化效果见第 4 章。

3.1 优化解释器翻译性能

3.1.1 背景介绍

完成 LoongArch 指令集架构的 Java 虚拟机后，对比 LoongArch 和 MIPS 架构处理器 Java 虚拟机纯解释器翻译性能。由于 LoongArch 架构处理器的 Java 虚拟机目前没有实现基础扩展指令的支持，为了使 MIPS 和 LoongArch 的环境保持一致，需要在 MIPS 指令集架构的 Java 虚拟机上禁用龙芯基础扩展指令 LoongEXT。MIPS 龙芯基础扩展指令 LoongEXT [3]主要用于提升处理器在访存、页表管理、X86 及 ARM 二进制翻译等方面的执行效率。通过使用 Java 虚拟机参数-XX:-UseLEXT1 -XX:-UseLEXT2 -XX:-UseLEXT3 可以禁用 MIPS 龙芯基础扩展指令 LoongEXT。

Java 虚拟机中总共有 235 个字节码，LoongArch 与 MIPS 所有字节码通过模板解释器生成的指令条数见表 3.1，其中 code size 是指 Java 虚拟机执行 java -XX:+PrintInterpreter 过程中所生成代码的大小。表 3.2 展示了 LoongArch 与 MIPS 字节码对应指令条数比较。结合表 3.1 和表 3.2，LoongArch 解释器翻译性能总体略高于 MIPS，但差距并不明显；而且还有将近 22%的字节码，LoongArch 的指令生成条数大于 MIPS。

为了找出 LoongArch 字节码对应指令条数大于 MIPS 的原因，接下来将以字节码 istore_0[18]为例进行分析。字节码 istore_0 的含义是将操作数栈栈顶的数存储到本地变量表中 0 地址处。表 3.3 展示了字节码 istore_0 分别在 LoongArch 和 MIPS 指令集架构对应的汇编指令。LoongArch 较 MIPS 多 4 条指令，对比两者生成指令的区别，发现由于 MIPS 的算数运算类指令 daddiu 能完

成 16 位立即数运算，而 LoongArch 指令 addi.d 只能完成 12 位的立即数运算，而 10240 需要 14 位的立即数空间，因此 LoongArch 需要借助 li 宏指令。li 宏指令加载立即数的代码见图 3.1，其通过 lu12i.w+ori+lu32i.d+lu52i.d 四条指令将 64 位立即数加载到寄存器中。因此对于 istore_0 字节码 LoongArch 较 MIPS 产生指令膨胀。

表 3.1 解释器翻译性能对比

Table 3.1 Interpreter translation performance comparison

评价指标	LoongArch	MIPS	降低率
汇编指令条数	17476 条	17984 条	2.82%
code size	158KB	163KB	3.07%

表 3.2 LoongArch 与 MIPS 字节码对应指令条数比较

Table 3.2 Comparison of the number of instructions corresponding to LoongArch and MIPS bytecode

字节码对应指令条数比较	个数	百分比
LoongArch > MIPS	51	21.70%
LoongArch = MIPS	95	40.43%
LoongArch < MIPS	89	37.87%
合计	235	100%

```
void MacroAssembler::li(Register rd, long imm) {
    lu12i_w(rd, split_low20(imm >> 12));
    ori(rd, rd, split_low12(imm));
    lu32i_d(rd, split_low20(imm >> 32));
    lu52i_d(rd, rd, split_low12(imm >> 52));
}
```

图 3.1 原始立即数加载实现

Figure 3.1 Original implementation of loading immediate number

表 3.3 字节码 istore_0 对应汇编指令

Table 3.3 Bytecode istore_ 0 corresponds to assembly instruction

LoongArch	MIPS

ld.w $a0,$sp,0 // 取操作数
addi.d $sp,$sp,8(0x8)
st.w $a0,$s7,0 // 执行
// 取下一个字节码
ld.bu $s1,$s0,1(0x1)
addi.d $s0,$s0,1(0x1)
// 指令分派
slli.d $t2,$s1,0x3
lu12i.w $t3,2(0x2)
ori $t3,$t3,0x800
lu32i.d $t3,0
lu52i.d $t3,$t3,0
add.d $t3,$s8,$t3
add.d $t3,$t2,$t3
ld.d $t3,$t3,0
jirl $zer o,$t3,0
0x00000000
0x00000000
	lw v0,0(sp) // 取操作数
	daddiu sp,sp,8
	sw v0,0(s7) // 执行
	// 取下一个字节码
	lbu s1,1(s0)
	daddiu s0,s0,1
	// 指令分派
	dsll t2,s1,0x3
	daddiu t3,gp,10240
	daddu t3,t2,t3
	ld t3,0(t3)
	jr t3
	nop
	nop

共计 16 条指令	共计 12 条指令

```
void MacroAssembler::li(Register rd, long imm) {
    if (imm <= max_jint && imm >= min_jint) {
        if (is_simm12(imm)) {
            addi_d(rd, R0, imm);          // 12 位
        } else {
            lu12i_w(rd, split_low20(imm >> 12));     // 32 位
            if (split_low12(imm))
                ori(rd, rd, split_low12(imm));
        }
    } else if (is_simm(imm, 52)) {                    //   52 位
        lu12i_w(rd, split_low20(imm >> 12));
        if (split_low12(imm))
            ori(rd, rd, split_low12(imm));
        lu32i_d(rd, split_low20(imm >> 32));
    } else {
        li64(rd, imm);       // 64 位
}
```

图 3.2 立即数加载优化实现

Figure 3.2 Optimized implementation of loading immediate number

3.1.2 立即数加载优化

原始立即数加载实现简洁，最大可加载 64 位立即数到寄存器。但是该实现没有考虑不同位数立即数的加载情况，如 12 位立即数加载仅用 addi.d 一条指令即可实现，32 位立即数加载也可仅靠 lu12i.w+ori 两条指令实现即可。因此，对不同位数的立即数分类考虑，优化后的立即数加载实现见图 3.2。

经过立即数加载优化实现后，字节码 istore_0 对应汇编指令条数从 16 条减到 14 条，汇编指令条数减少 12.5%，见表 3.7。

3.1.3 使用移位运算指令和双寄存器访存指令优化指令分派

解释器执行包括四个步骤：

1. 取操作数：获取该字节码所需的操作数。
2. 执行：执行该字节码的处理例程。
3. 取下一个字节码。
4. 指令分派：获取下一个需要执行字节码的入口地址。

相关研究表明，指令分派部分占据了解释器执行的大部分时间，对解释器的性能影响很大[25]。若能减少指令分派所需要的指令，即可大大增加解释器的运行性能。指令分派的具体实现见 2.3.3 节。

以 table_offset 是 12 位立即数为例，原始指令分派实现需要 1 条移位指令、2 条加法指令和 1 条访存指令，总共 4 条指令才能获取下一个字节码的入口地址。LoongArch 指令集新增移位运算指令 alsl.d 和双寄存器访存指令 ldx.d，指令具体含义和用法分别见表 3.4 和表 3.5。运用这两条指令，指令分派仅需要一条移位运算指令 alsl.d 和一条双寄存器访存指令 ldx.d，总共 2 条指令即可获取下一个字节码的入口地址。

表 3.4 移位运算指令

Table 3.4 Shift operation instruction

alsl.d rd, rj, rk, sa2 // 将通用寄存器 rj 中的[63:0]位数据逻辑左移（sa2+1）位后加上通用寄存器 rk 中的[63:0]位数据，所得结果写入通用寄存器 rd 中。

表 3.5 双寄存器访存指令

Table 3.5 Dual register memory access instruction

ldx.d rd, rj, rk // 从内存 rj+rk 处取回一个双字的数据符号扩展后写入通用寄存器 rd。

优化后的 dispatch_base()方法核心代码见表 3.6。通过使用移位运算指令和双寄存器访存指令优化指令分派，字节码 istore_0 对应的汇编指令从 3.1.2 节的 14 条精简到 12 条，相比优化前的 16 条汇编指令，汇编指令条数减少 25%，见表 3.7。

表 3.6 dispatch_base()方法优化前后对比

Table 3.6 Comparison of dispatch_base() methods before and after optimization

优化前	优化后

slli_d(T2, Rnext, LogBytesPerWord);
if(table_offset != 0) {
  if (is_simm12(table_offset)) {
  addi_d(T3, S8, table_offset);
  }
  else {
  li(T3, table_offset);
  add_d(T3, S8, T3);
  }
  add_d(T3, T2, T3);
  ld_d(T3, T3, 0);
}
else {
  add_d(T3, T2, S8);
  ld_d(T3, T3, 0);
}
	if(table_offset != 0) {
	  if (is_simm12(table_offset)) {
	  alsl_d(T3, Rnext, S8, LogBytesPerWord - 1);
	  ld_d(T3, T3, table_offset);
	  }
	  else {
	  li(T2, table_offset);
	  alsl_d(T3, Rnext, S8, LogBytesPerWord - 1);
	  ldx_d(T3, T2, T3);
	  }
	}
	else {
	  slli_d(T2, Rnext, LogBytesPerWord);
	  ldx_d(T3, S8, T2);
	}

3.1.4 小结

将 Java 虚拟机移植到 LoongArch 指令集架构之后，通过测试发现 LoongArch 架构的 Java 虚拟机解释器翻译性能较 MIPS 提升并不大，仅有 3%左右，和预期有一定的差距。通过对比字节码在两个架构生成的指令，首先对立即数加载进行了优化，使得 Java 虚拟机能够根据立即数的位数采取不同的指令条数，减少了立即数加载的指令条数；接着使用 LoongArch 指令集架构中的移

位运算指令和双寄存器访存指令优化指令分派，进一步减少了指令条数。具体的静态数据和启动性能效果见 4.2 节。

表 3.7 字节码 istore_0 优化前后对比

Table 3.7 Comparison of bytecode istore_0 before and after optimization

优化前	立即数加载优化	移位运算指令和 双寄存器访存优化
ld.w $a0,$sp,0 addi.d $sp,$sp,8(0x8) st.w $a0,$s7,0 ld.bu $s1,$s0,1(0x1) addi.d $s0,$s0,1(0x1) slli.d $t2,$s1,0x3 lu12i.w $t3,2(0x2) ori $t3,$t3,0x800 lu32i.d $t3,0 lu52i.d $t3,$t3,0 add.d $t3,$s8,$t3 add.d $t3,$t2,$t3 ld.d $t3,$t3,0 jirl $zero,$t3,0 0x00000000 0x00000000	ld.w $a0,$sp,0 addi.d $sp,$sp,8(0x8) st.w $a0,$s7,0 ld.bu $s1,$s0,1(0x1) addi.d $s0,$s0,1(0x1) slli.d $t2,$s1,0x3 lu12i.w $t3,2(0x2) ori $t3,$t3,0x800 add.d $t3,$s8,$t3 add.d $t3,$t2,$t3 ld.d $t3,$t3,0 jirl $zero,$t3,0 0x00000000 0x00000000	ld.w $a0,$sp,0 addi.d $sp,$sp,8(0x8) st.w $a0,$s7,0 ld.bu $s1,$s0,1(0x1) addi.d $s0,$s0,1(0x1) lu12i.w $t3,2(0x2) ori $t3,$t3,0x800 alsl.d $t3,$s1,$s8,0x3 ldx.d $t3,$t2,$t3 jirl $zero,$t3,0 0x00000000 0x00000000
共计 16 条指令	共计 14 条指令	共计 12 条指令

3.2 跳转机制的优化方法

3.2.1 跳转机制

本小节首先对 LoongArch 跳转相关指令做简要的介绍，接着介绍了实现 Java 虚拟机跳转机制基础的 LoongArch Label 类，最后介绍了原始的跳转实现机制。

3.2.1.1 LoongArch 跳转指令介绍

跳转指令分为两种，一种是 PC 相对地址跳转指令，相应的助记符以 b 开头；另一种是绝对地址跳转指令，相应的助记符以 j 开头[19]。跳转指令根据是否有条件，又可分为条件跳转指令和无条件跳转指令。LoongArch 中的条件跳转指令有 beq/bne/blt/bltu/bge/bgeu 等；无条件跳转指令包括 b/bl/jirl 指令。

LoongArch 的所有跳转指令见表 3.8，其中的 imm16,imm21,imm26 分别表示 16 位、21 位、26 位立即数；rj,rd 表示寄存器。

表 3.8 LoongArch 跳转指令

Table 3.8 Jump instructions of LoongArch

指令	跳转范围	跳转指令类型
beq/bne/blt/bge/bltu/bgeu rj, rd, imm16	[PC-128KB, PC+128KB]	相对地址跳转 有条件跳转
beqz/bnez rj, imm21	[PC-4MB, PC+4MB]	相对地址跳转 有条件跳转
b/bl imm26	[PC-128MB, PC+128MB]	相对地址跳转 无条件跳转
jirl rd, rj, imm16	64 位全地址空间	绝对地址跳转 无条件跳转

3.2.1.2 Label 类

LoongArch Label 类是实现 Java 虚拟机跳转机制的基础，它表示跳转指令的目标。Java 虚拟机中的宏汇编跳转指令将 Label 对象作为它们的一个参数。Label 对象有两种状态，分别是 bound 状态和 unbound 状态。所有的 Label 对象初始化时均为 unbound 状态，通过汇编器中的 bind 方法，为 Label 对象绑定跳转的目标地址，Label 对象即从 unbound 状态切换成 bound 状态。当编译某条跳转指令时，若该指令的 Label 对象为 bound 状态，即已知跳转的目标地址，将该地址填入指令即可。若该指令的 Label 对象为 unbound 状态，即还不知道跳转的目标地址，此时先将 0 填入指令，等到该 Label 对象通过 bind 方法绑定跳转的目标地址之后，再将该目标地址写入。这种之后再回填目标地址的方式称为 patch。patch 有一个最大的特点是，由于编译时不清楚需要填入值的大小，因此需要考虑该值所有可能的情况，即需要给该值预留一个最大的空间。表 3.9 中的代码取自 Java 虚拟机中的宏汇编指令 bne_long，该段代码展示了 Label 的 bound 状态和 unbound 状态。

表 3.9 LoongArch Label 的 bound 状态和 unbound 状态

Table 3.9 bound LoongArch Label and unbound LoongArch Label

```
void MacroAssembler::bne_long(Register rs, Register rt, LoongArch Label& L) {
   LoongArch Label not_taken;        // LoongArch Label not_taken 声明，初始化，ubound 状态
   beq(rs, rt, not_taken);        // ubound 状态，需要 patch
   jmp_far(L);
   bind(not_taken);        // bound 状态，将目标偏移地址回填
}
```

3.2.1.3 原始跳转机制实现

Java 虚拟机中将跳转机制分为长跳转和短跳转两类。短跳转用 b/bl 指令实现，b/bl 指令是相对地址跳转指令，跳转范围在相对 PC 的 128MB 范围内；长跳转用 jirl 指令实现，jirl 指令是绝对地址跳转指令。与 b/bl 指令跳转偏移地址直接在指令编码不同，jirl 指令跳转目标地址是通过寄存器传入，因此在调用 jirl 指令之前需要将跳转目标地址写入寄存器中。通过 lu12i.w+lu32i.d+lu52i.d 三条指令，即可拼凑出一个 64 位数字，因此 jirl 指令的跳转范围为全 64 位地址空间。表 3.10 展示了短跳转和长跳转的实现方式，其中 jirl rd, ra, imm16（imm10）是指 jirl 指令编码中有 16 位是立即数，但是在本实现中只用低 10 位。图 3.3 展示了 lu12i.w+lu32i.d+lu52i.d 三条指令拼凑出的 64 位数字。

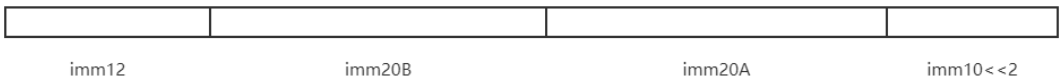

图 3.3 长跳转指令拼接的 64 位立即数

Figure 3.3 64 bit immediate of long jump instruction splicing

表 3.10 原始跳转机制实现

Table 3.10 Original jump implementation

跳转类型	指令	跳转范围
短跳转	b/bl imm26	[PC-128MB, PC+128MB]
长跳转	lu12i.w ra, imm20A lu32i.d ra, imm20B lu52i.d ra, ra, imm12 jirl rd, ra, imm16（imm10）	全 64 位地址空间

3.2.2 使用相对 PC 运算指令实现跳转机制

原始跳转机制的实现没有充分考虑 LoongArch 指令集的特点，LoongArch 指令集中有相对 PC 运算指令 pcaddu12i 和 pcaddu18i，这两条指令具体的介绍见表 3.11。将 pcaddu12i 和 pcaddu18i 这两条指令和 jirl 指令配合使用，即可组成跳转范围更大的相对 PC 跳转指令，见表 3.12。表 3.12 中 jirl rd, ra, imm16(imm18)是指 jirl 指令编码中立即数的位数为 16 位，但是实际上 jirl 指令发射时是将 16 位立即数左移两位，即 18 位的立即数。这样就可以和 pcaddu18i 指令编码中的 20 位立即数一起拼接成一个 38 位的立即数。所以 pcaddu18i+jirl 指令的跳转范围为[PC-128GB, PC+128GB]。jirl rd, ra, imm16(imm10)同理，就不在此一一赘述了。

表 3.11 pcaddu12i 和 pcaddu18i 指令

Table 3.11 pcaddu12i and pcaddu18i instructions

pcaddu12i rd, imm20 // pcaddu12i 将 20 比特立即数 imm20 最低位连接上 12 比特 0 之后符号扩展，所得数据加上该指令的 PC，相加结果写入通用寄存器 rd 中。
pcaddu18i rd, imm20 // pcaddu18i 将 20 比特立即数 imm20 最低位连接上 18 比特 0 之后符号扩展，所得数据加上该指令的 PC，相加结果写入通用寄存器 rd 中。

表 3.12 使用相对 PC 运算指令实现跳转机制

Table 3.12 Implementation of jump mechanism using reLoongArchtive PC operation instructions

指令	跳转范围
pcaddu12i ra, imm20; jirl rd, ra, imm16(imm10)	[PC-2GB, PC+2GB]
pcaddu18i ra, imm20; jirl rd, ra, imm16(imm18)	[PC-128GB, PC+128GB]

使用相对 PC 运算指令实现跳转机制的好处在于：原始跳转机制实现中相对 PC 跳转范围为[PC-128MB, PC+128MB]，将 PC 相对运算指令和 jirl 指令结合使得相对 PC 跳转范围大大增大。同时相对地址跳转与绝对地址跳转相比是有优势的，因为绝对地址跳转需要将跳转的目标地址写入寄存器，而跳转的目标地址一般是很大的，需要多条算数运算类指令才可以拼接成目标地址；但是相对地址跳转指令是将跳转目标相较于当前 PC 的偏移值写入指令编码，这个值相比跳转目标的绝对地址是大大减小的。

3.2.3 调整 CodeCache 虚拟内存映射空间

- CodeCache

CodeCache 又叫代码缓存，Java 虚拟机生成的所有代码都放在 CodeCache 里。与 CodeCache 相关的 Java 虚拟机参数有三个，分别是 ReservedCodeCacheSize, InitialCodeCacheSize, CodeCacheExpansionSize。ReservedCodeCacheSize 表示 Java 虚拟机向虚拟内存申请预留给 CodeCache 的大小，即 CodeCache 最大的容量。InitialCodeCacheSize 表示 CodeCache 初始状态时的大小。CodeCacheExpansionSize 指 CodeCache 每一次扩展的大小，即当当前分配的 CodeCache 被填满后，向 ReservedCodeCache 继续申请 CodeCacheExpansionSize 大小的容量。

ReservedCodeCacheSize, InitialCodeCacheSize 和 CodeCacheExpansionSize 的大小是和架构相关的，在 LoongArch 架构中它们的默认值分别为 48MB，2496KB，64KB。ReservedCodeCacheSize 的大小可以使用以下这个虚拟机开关

来控制 CodeCache 的最大值：

-XX: ReservedCodeCacheSize=<n>

n 的最大值不能超过 2GB，在当前 Java 虚拟机版本，当 n 超过 2GB 时，Java 虚拟机将不再提供支持。

- CodeCache 和 libjvm.so 虚拟内存映射

Java 虚拟机中的跳转机制涉及到两个层面，一种是 CodeCache 内部之间的跳转，一种是从 CodeCache 跳转到 libjvm.so 中。libjvm.so 是 Java 虚拟机编译生成的共享库，大小一般为几十 MB。CodeCache 在 LoongArch 的默认值为 48MB，最大值为 2GB。CodeCache 和 libjvm.so 的相对位置决定了 Java 虚拟机跳转机制的范围。图 3.5 为 Java 线程运行时的一个虚拟内存映射。通过在 Java 虚拟机中加打印函数得知 CodeCache 在虚拟内存的范围区间为[0x00014000, 0x03014000]。

Java 虚拟机共享库位于较高的虚拟地址空间，这是 linux 内核的虚拟内存空间映射机制决定的。48MB 的 CodeCache 位于[0x00014000, 0x03014000]范围区间。这是通过 share/vm/runtime/virtualspace.cpp 中的 ReservedSpace::initialize 函数确定的，代码见图 3.4。通过图 3.4 中代码可以看到，ReservedSpace::initialize 函数中对 MIPS 架构和 LoongArch 架构处理器的 CodeCache 映射做了特殊的处理。这两个架构处理器的 CodeCache 映射空间的起始地址位于距虚拟内存起始地址 0x0 5 个页大小的位置，LoongArch 每个页的大小为 16KB，5 个页的大小即为 80KB，因此 CodeCache 的起始地址即为 0x14000。LoongArch 架构上 CodeCache 的默认大小为 48MB，因此 CodeCache 的范围区间为[0x00014000, 0x03014000]。这和上述打印出来的 CodeCache 范围区间一致。

与 MIPS 架构和 LoongArch 架构在距离虚拟内存起始地址 0x0 5 个页大小的位置申请 CodeCache 空间不同的是，其他主流架构 CodeCache 的虚拟内存映射空间均是通过内核的虚拟内存空间映射机制决定的。因此 CodeCache 在虚拟内存映射的位置虽然每一次随着运行程序的不同而有所不同，但是 CodeCache 的映射位置总是和共享库的位置是相邻的。

原始跳转机制实现中只有跳转范围为[PC-128MB, PC+128MB]的相对地址跳转和 64 位全地址空间跳转。64 位全地址空间跳转采用的指令是 jirl 指令，跳转

的目标地址是存储在寄存器里，因此在调用 jirl 指令之前需要若干条立即数加载指令，立即数加载指令的条数和该立即数的大小有关。CodeCache 的最大大小为 2GB，libjvm.so 共享库的大小为几十 MB，显然相对地址跳转指令 b/bl 的跳转范围是不够的，因此就得采用绝对地址跳转。而 libjvm.so 和 CodeCache 的地址在虚拟内存的高地址空间，需要耗费的立即数加载指令的条数较多，因此对 LoongArch 架构和 MIPS 架构处理器 Java 虚拟机的 CodeCache 进行了特殊化处理，强制 CodeCache 分配在低地址空间，有效降低了立即数加载指令的条数。

```
void ReservedSpace::initialize(size_t size, size_t alignment, bool LoongArchrge,
                                        char* requested_address,
                                        const size_t noaccess_prefix,
                                        bool executable) {
...
...
   char* base = NULL;

#if (defined MIPS || defined LOONGARCH) && !defined ZERO
   // LoongArch 架构上 os::Linux::page_size()=16K,因此 opt_reg_addr=80K
   size_t opt_reg_addr = 5 * os::Linux::page_size();
   static int code_cache_init_fLoongArchg = 1;
   if (UseCodeCacheAllocOpt && code_cache_init_fLoongArchg && executable) {
     code_cache_init_fLoongArchg = 0;
     //  由于 opt_reg_addr=80K，因此 requested_address=0X14000
      requested_address = (char*) opt_reg_addr;
   }
#endif
…
}
```

图 3.4 ReservedSpace::initialize 代码片段

Figure 3.4 Code fragment of reservedspace:: initialize

```
$ cat /proc/10898/maps
00010000-00014000 r-xp 00000000 00:00 0
// CodeCache 范围区间  [0x00014000, 0x03014000]
00014000-00284000 rwxp 00000000 00:00 0        // InitialCodeCache=4096K
00284000-03014000 ---p 00000000 00:00 0        //  待扩展的 CodeCache 区间
...

aaab67c000-aaab680000 r-xp   j2sdk   -image/bin/java            // java 线程
aaab680000-aaab684000 r-xp   j2sdk   -image/bin/java
aaab684000-aaab688000 rwxp   j2sdk   -image/bin/java
aab6550000-aab6574000 rwxp   [heap]
...
...
//  其他共享库
fff5dc0000-fff5ddc000 r-xp     j2sdk     -image/jre/lib/LoongArch64/libzip.so
fff5ddc000-fff5de0000 r-xp     j2sdk     -image/jre/lib/LoongArch64/libzip.so
fff5de0000-fff5de4000 rwxp    j2sdk    -image/jre/lib/LoongArch64/libzip.so
…
…
// libjvm.so 共享库
fff601c000-fff6a78000 r-xp      j2sdk-image/jre/lib/LoongArch64/server/libjvm.so
fff6a78000-fff6ad8000 r-xp   j2sdk   -image/jre/lib/LoongArch64/server/libjvm.so
fff6ad8000-fff6b08000 rwxp   j2sdk   -image/jre/lib/LoongArch64/server/libjvm.so
```

图 3.5 Java 线程运行时虚拟内存映射

Figure 3.5 Virtual memory mapping of java thread runtime

在 3.2.2 节中，使用相对 PC 运算指令实现跳转机制对 LoongArch 架构 Java 虚拟机的跳转机制进行了优化，使得相对地址跳转的范围从[PC-128MB, PC+128MB]上升到了[PC-128GB, PC+128GB]。[PC-128GB, PC+128GB]的范围

空间对于 2GB 的 CodeCache 和几十 MB 的 libjvm.so 共享库是完全足够的。此时再将 CodeCache 分配在低地址空间没有实际意义，同时还会拉远 CodeCache 和 libjvm.so 共享库的距离。因此不再对 LoongArch 架构 Java 虚拟机的 CodeCache 进行特殊化处理，强制 CodeCache 分配在低地址空间，而是采取和其他主流架构一样的虚拟内存映射方式，使得 CodeCache 的映射位置总是和共享库的位置是相邻，见表 3.13。

表 3.13 Java 线程运行时虚拟内存映射（新）

Table 3.13 Virtual memory mapping of java thread runtime (NEW)

```
$ cat /proc/2549/maps
00010000-00014000 r-xp 00000000 00:00 0
6cb000000-6d5580000 rwxp 00000000 00:00 0
...
aaad350000-aaad354000 r-xp          j2sdk-image/bin/java     // Java 线程
aaad354000-aaad358000 r-xp                j2sdk-image/bin/java
aaad358000-aaad35c000 rwxp                 j2sdk-image/bin/java
…
...
// CodeCache 范围区间  [0x fff0204000, 0x fff3204000]
fff0204000-fff0474000 rwxp 00000000 00:00 0     // InitialCodeCache=4096KB
fff0474000-fff3204000 ---p 00000000 00:00 0     //  待扩展的 CodeCache 区间
//  其他共享库
fff3204000-fff3220000 r-xp          j2sdk-image/jre/lib/LoongArch64/libzip.so
fff3220000-fff3224000 r-xp          j2sdk-image/jre/lib/LoongArch64/libzip.so
fff3224000-fff3228000 rwxp           j2sdk-image/jre/lib/LoongArch64/libzip.so
…
…
// libjvm.so 共享库
fff3460000-fff3ebc000 r-xp      j2sdk-image/jre/lib/LoongArch64/server/libjvm.so
fff3ebc000-fff3f1c000 r-xp       j2sdk-image/jre/lib/LoongArch64/server/libjvm.so
fff3f1c000-fff3f4c000 rwxp       j2sdk-image/jre/lib/LoongArch64/server/libjvm.so
```

3.2.4 跳转机制优化实现

通过优化 CodeCache 虚拟内存映射空间和使用相对 PC 运算指令实现跳转机制，优化后的跳转机制见表 3.14。跳转机制优化前后的对比见表 3.15，优化后的跳转机制与原始跳转机制相比主要有以下两个优点：一、简化了跳转机制的实现。从表 3.15 中看起来可能差距不大，只有两条指令的差距。但是实际上 Java 虚拟机的跳转机制不仅仅只涉及到指令条数的变化。patch 时需要考虑最坏的情况，预留指令代码空间，直到确定跳转的目标地址再将其写入预留的代码空间。原始跳转实现，patch 时需要考虑好 4 种情况，见表 3.16，而优化后的跳转机制实现只需要简单的考虑 2 种情况即可，见表 3.17。二、消除了绝对地址跳转，全都采用相对地址跳转实现。用户代码空间在高地址空间，这意味着需要若干条加载指令才能将该地址写入跳转指令的寄存器，而采用相对地址跳转即可避免这种情况。

表 3.14 跳转机制优化实现

Table 3.14 Optimized implementation of jump mechanism

跳转类型	指令	跳转范围
短跳转	b/bl imm26	[PC-128MB, PC+128MB]
长跳转	pcaddu18i ra, imm20; jirl rd, ra, imm16(imm18)	[PC-128GB, PC+128GB]

表 3.15 跳转机制实现对比

Table 3.15 Implementation comparison of jump mechanism

	优化前	优化后
短跳转	b/bl imm26	b/bl imm26
长跳转	lu12i.w ra, imm20A lu32i.d ra, imm20B lu52i.d ra, ra, imm12 jirl rd, ra, imm16 (imm10)	pcaddu18i ra, imm20; jirl rd, ra, imm16(imm18)
跳转类型	短跳转：相对地址跳转 长跳转：绝对地址跳转	短跳转：相对地址跳转 长跳转：相对地址跳转

表 3.16 原始跳转机制 patch 情况分类

Table 3.16 Patch classification of original jump mechanism

情况	指令
跳转的目标地址距离分支指令在 128M 的范围内	b/bl imm26 nop nop nop
跳转目标的绝对地址在 32 位地址空间	lu12i.w ra, imm20A jirl rd, ra, imm16 (imm10) nop nop
跳转目标的绝对地址在 52 位地址空间	lu12i.w ra, imm20A lu32i.d ra, imm20B jirl rd, ra, imm16 (imm10) nop
跳转目标的绝对地址在 64 位地址空间	lu12i.w ra, imm20A lu32i.d ra, imm20B lu52i.d ra, ra, imm12 jirl rd, ra, imm16 (imm10)

表 3.17 跳转优化机制 patch 情况分类

Table 3.17 Patch classification of jump optimization mechanism

情况	指令
跳转的目标地址距离分支指令在 128M 的范围内	b/bl imm26 nop
跳转的目标地址距离分支指令在 128G 的范围内	pcaddu18i ra, imm20; jirl rd, ra, imm16(imm18)

3.2.5 小结

原始跳转机制的实现没有充分考虑 LoongArch 指令集的特点，LoongArch 指令集中有相对 PC 运算指令。利用相对 PC 运算指令，结合优化 CodeCache 内存布局，使得 128GB 范围的相对地址跳转能够覆盖 LoongArch 64 位 Java 虚拟机的所有跳转情况。这一优化不仅减少了实现跳转机制的指令，同时消除了绝对地址跳转。具体的性能效果见 4.3 节。

3.3 条件传送机制的优化方法

3.3.1 背景介绍

完成 LoongArch 指令集架构的 Java 虚拟机后，采用 SPECjvm2008 测试工具对其进行性能测试，并与 MIPS 指令集架构的 Java 虚拟机性能测试结果进行对比，发现 LoongArch 指令集架构的 Java 虚拟机在 scimark.monte_carlo 测试程序上与 MIPS 指令集架构的 Java 虚拟机有 30%以上的性能差距，表 3.18 列出了 LoongArch 和 MIPS 在 scimark.monte_carlo 测试程序上的得分，分值越高代表性能越好。

表 3.18 scimark.monte_carlo 测试结果

Table 3.18 scimark.monte_carlo test result

Benchmark	LoongArch (ops/m)	MIPS (ops/m)	增长率
scimark.monte_carlo	78.22	115.31	-32.17%

30%以上的性能差距是不合常理的。为了找出 LoongArch 架构 Java 虚拟机性能较低的原因，做了如下实验：

1. Java 虚拟机默认启动参数。通过使用 Java 虚拟机参数-XX:+PrintFlagsFinal，打印 LoongArch 和 MIPS 架构 Java 虚拟机默认启动参数，两者没有区别。
2. 多线程。通过使用 SPECjvm2008 参数-bt 指定测试程序在单线程下运行(-bt 1)，MIPS 和 LoongArch 关于 scimark.monte_carlo 测试程序的得分分别为 28.43 和 18.18，LoongArch 较 MIPS 仍有 30%以上性能的差距。

3. 编译线程。通过使用 SPECjvm2008 参数-wt 加大执行线程的时间占比，LoongArch 与 MIPS 得分较默认情况基本相同。
4. MIPS 龙芯基础扩展指令 LoongEXT。禁用后 MIPS 指令集架构的 Java 虚拟机在 scimark.monte_carlo 测试程序上的得分为 106.11，相较默认情况性能下降 8.67%。
5. 解释器和 C2 编译器。通过使用 Java 虚拟机参数-Xint 强制 Java 虚拟机在纯解释器模式下运行，MIPS 和 LoongArch 关于 scimark.monte_carlo 测试程序的得分分别为 46.62 和 47.81，即在纯解释器下 LoongArch 性能较 MIPS 更好。通过使用 Java 虚拟机参数-Xcomp 使 Java 虚拟机在纯翻译模式下运行，MIPS 和 LoongArch 关于 scimark.monte_carlo 测试程序的得分分别为 128.37 和 76.48，LoongArch 较 MIPS 有 30%以上的差距。
6. 热点方法。通过使用 Java 参数-Xprof 获取程序热点函数，MIPS 和 LoongArch 的热点方法均为 MonteCarlo::integrate，该函数输入为样本点的个数，输出为 MonteCarlo 算法估计的∏值。
7. 分支预测率。通过 perf 工具获取了 LoongArch 和 MIPS 关于 scimark.monte_carlo 测试程序的分支预测率，分别为 2.99%和 0.02%，LoongArch 的分支预测率是 MIPS 的 150 倍。

通过实验结果，排除了 Java 虚拟机默认启动参数、编译线程、多线程对 scimark.monte_carlo 测试结果的影响。MIPS 龙芯基础扩展指令 LoongEXT 对性能有 9%左右的提升，而 LoongArch 的 Java 虚拟机目前没有实现基础扩展指令的支持，因此后续为了更好地分析性能差距的原因，需要在 MIPS 指令集架构的 Java 虚拟机上禁用龙芯基础扩展指令 LoongEXT。分析纯解释和纯翻译模式下的测试结果，可知影响 scimark.monte_carlo 测试结果的原因在 C2 编译器上。

因此接下来，利用 perf 分别采集 LoongArch 和 MIPS 运行 scimark.monte_carlo 测试程序的性能数据，同时使用 Java 虚拟机参数-XX:+PrintCompilation 和-XX:+PrintOptoAssembly 打印出 C2 编译器生成 MonteCarlo::integrate 方法的汇编指令。LoongArch 和 MIPS 的 scimark.monte_carlo 程序 perf 性能采集结果分别见图 3.6 和图 3.7，LoongArch 和 MIPS C2 编译器生成 MonteCarlo::integrate 方法的部分汇编指令见表 3.19 和

表 3.20。

```
Samples: 23K of event 'task-clock:u', Event count (approx.): 5806000000
Overhead  Command          Shared Object        Symbol
  20.09%  java             perf-25523.map       [.] 0x000000ffe9295374
  15.29%  java             perf-25523.map       [.] 0x000000ffe9295378
   5.16%  java             perf-25523.map       [.] 0x000000ffe9295390
   3.04%  VM Thread        libc-2.28.so         [.] __sched_yield
   2.95%  java             perf-25523.map       [.] 0x000000ffe9295400
   2.71%  java             perf-25523.map       [.] 0x000000ffe92953b4
   2.59%  java             perf-25523.map       [.] 0x000000ffe92953a0
   2.45%  java             perf-25523.map       [.] 0x000000ffe929533c
   2.40%  java             perf-25523.map       [.] 0x000000ffe92953a4
   2.36%  java             perf-25523.map       [.] 0x000000ffe92953d4
```

图 3.6 LoongArch 的 scimark.monte_carlo 程序 perf 性能采集结果

Figure 3.6 LoongArch perf performance results of scimark.monte_carlo

```
Samples: 33K of event 'task-clock', Event count (approx.): 8359500000
Overhead  Command          Shared Object        Symbol
  13.50%  java             perf-29089.map       [.] 0x00000000002b60b8
  10.17%  java             perf-29089.map       [.] 0x00000000002b60bc
   4.41%  java             perf-29089.map       [.] 0x00000000002b60c4
   2.05%  java             perf-29089.map       [.] 0x00000000002b604c
   1.97%  java             perf-29089.map       [.] 0x00000000002b603c
   1.95%  java             perf-29089.map       [.] 0x00000000002b601c
   1.93%  java             perf-29089.map       [.] 0x00000000002b602c
   1.81%  java             perf-29089.map       [.] 0x00000000002b6014
   1.75%  java             perf-29089.map       [.] 0x00000000002b5fcc
   1.75%  java             perf-29089.map       [.] 0x00000000002b6010
   1.74%  java             perf-29089.map       [.] 0x00000000002b6038
   1.74%  java             perf-29089.map       [.] 0x00000000002b6054
   1.74%  java             perf-29089.map       [.] 0x00000000002b605c
```

图 3.7 MIPS 的 scimark.monte_carlo 程序 perf 性能采集结果

Figure 3.7 MIPS perf performance results of scimark.monte_carlo

比较 LoongArch 和 MIPS 的热点指令，发现在 LoongArch 中排名第一热点最高的指令为地址 0x000000ffe9295374 处的 move 指令，同时排名第二的指令为其下一条 slt 指令，这两条指令运行时间占程序总共运行时间的 35.38%。而 MIPS 中对应处的指令则为地址 0x00000000002b6014 处的条件传送指令 movt 指令，且其运行时间仅占程序总共运行时间的 1.81%。LoongArch 指令集中没有条件传送指令，LoongArch 架构的 Java 虚拟机中是将 movt 指令翻译成了 bceqz 和 move 这 2 条指令。这也很好地解释了为什么 scimark.monte_carlo 程序中 LoongArch 较 MIPS 分支预测率高达 150 倍的原因。

修改 MIPS 的 ad 文件，禁用条件传送指令 movt，转用分支指令实现，

scimark.monte_carlo 程序的得分从 115.31 降为 75.63，性能下降幅度高达 34%，和 LoongArch 性能相当；若同时禁用龙芯扩展指令 LoongEXT，下降幅度高达 42%，较 LoongArch 性能低 8%。因此，可以断定影响 LoongArch 架构 Java 虚拟机性能的原因在于 LoongArch 指令集缺少条件传送指令。

表 3.19 LoongArch 部分汇编指令

Table 3.19 LoongArch partial assembly instructions

```
0x000000ffe9295364: fmul.d      $f3,$f3,$f3
0x000000ffe9295368: fadd.d      $f2,$f2,$f3
0x000000ffe929536c: fcmp.cult.d          $fcc0,$f1,$f2
0x000000ffe9295370: bceqz       $fcc0,8(0x8) # 0x000000ffe9295378
0x000000ffe9295374: move        $s7,$t2              ;*iinc
                                                     ; -
jnt.scimark2.MonteCarlo::integrate@50 (line 53)

0x000000ffe9295378: slt         $t7,$t3,$t1
0x000000ffe929537c: beq         $t7,$zero,168(0xa8) # 0x000000ffe9295424
                                                     ;*if_icmpge
```

表 3.20 MIPS 部分汇编指令

Table 3.20 MIPS partial assembly instructions

```
0x00000000002b6008: mul.d       $f3,$f3,$f3
0x00000000002b600c: add.d       $f2,$f2,$f3
0x00000000002b6010: c.ult.d     $f1,$f2
0x00000000002b6014: movt        s7,t2,$fcc0          ;*iinc
                                                     ; -
jnt.scimark2.MonteCarlo::integrate@50 (line 53)

0x00000000002b6018: slt         at,t3,t1
0x00000000002b601c: beqz        at,0x00000000002b60fc
0x00000000002b6020: nop                              ;*if_icmpge
```

3.3.2 MIPS 条件传送指令介绍

条件传送指令[19]只要相应的条件满足，就会把数据从一个寄存器传递到另一个寄存器中，否则就什么也不做。由于分支转移指令对于发挥流水线的效率非常不利，因此，条件传送指令的出现使得编译器能用更少的条件分支完成给

定的效率，从而提供了执行效率。

下面这段求两个数的最小值的代码：

n = (a < b) ? a : b;

假定编译器已经把所有要处理的变量放到了寄存器中，运用条件传送指令和分支指令的汇编代码如表 3.21 所示，条件传送指令与分支指令相比可以省掉一条跳转指令，因此具有更多级流水线的 CPU 就可以运转得更快了。

表 3.21 分支指令和条件传送指令

Table 3.21 Branch instruction and Conditional Move instruction

分支指令	条件传送指令

slt t0, a, b
move n, a
bne zero, t0, 1f
move n, b
1:
	slt t0, a, b
	move n, a
	movz n, b, t05

MIPS 中的条件传送指令可以分为四类，分别是整数条件整数传送（Integer Condition Integer Move，简称 ICIM）、浮点条件整数传送（Floating Condition Integer Move，简称 FCIM）、整数条件浮点传送（Integer Condition Floating Move，简称 ICFM）、浮点条件浮点传送（Floating Condition Floating Move，简称 FCFM），具体的用法和含义见表 3.23，其中 movz、movn、movf、movt 分别是 MIPS 机器指令；rd, rj, rk 分别是整数寄存器；fj, fd 分别是浮点寄存器；ca 为浮点的条件标志寄存器。

表 3.22 fsel 指令

Table 3.22 fsel instruction

fsel fd, fj, fk, ca // fsel 指令执行条件赋值操作。fsel 指令执行时，如果条件标志寄存器 ca 的值等于 0 则将浮点寄存器 fj 的值写入到浮点寄存器 fd 中，否则将浮点寄存器 fk 的值写入到浮点寄存器 fd 中。

表 3.23 MIPS 条件传送指令

Table 3.23 MIPS Conditional Move Instruction

类型	指令	含义

整数条件整数传送
ICIM
	movz rd, rj, rk	if (rk = 0)
		rd = rj;
		else
		rd = rd;

	movn rd, rj, rk	if (rk != 0)
		rd = rj;
		else
		rd = rd;

浮点条件整数传送
FCIM
	movz rd, rj, ca	if (ca = 0)
		rd = rj;
		else
		rd = rd;

	movn rd, rj, ca	if (ca != 0)
		rd = rj;
		else
		rd = rd;

整数条件浮点传送
ICFM
	movz fd, fj, rk	if (rk = 0)
		fd = fj;
		else
		fd = fd;

	movn fd, fj, rk	if (rk != 0)
		fd = fj;
		else
		fd = fd;

浮点条件浮点传送
FCFM
	movf fd, fj, ca	if (ca = 0)
		fd = fj;
		else
		fd = fd;

	movt fd, fj, ca	if (ca != 0)
		fd = fj;
		else
		fd = fd;

3.3.3 条件传送机制优化实现

3.3.3.1 原始条件传送机制实现

LoongArch 指令集中只有浮点条件浮点传送指令 fsel，其具体含义及用法见表 3.22，其他三种类型的传送指令在 LoongArch 指令集中均没有对应的机器指令，而是由分支指令和 move 指令结合起来实现。原始 LoongArch 条件传送机制见表 3.24，其中 fmov.{s/d} fd, fj 的含义是将浮点寄存器 fj 的值按单精度/双精度浮点数格式写入到浮点寄存器 fd 中，如果 fj 的值不是单精度/双精度浮点

数格式，则结果不确定。

表 3.24 原始条件传送机制

Table 3.24 LoongArch Original Condition Move

类型	指令	含义

整数条件整数传送
ICIM
	bnez rk, 1f
	move rd, rj
	1:
		if (rk = 0)
		rd = rj;
		else
		rd = rd;

	beqz rk, 1f
	move rd, rj
	1:
		if (rk != 0)
		rd = rj;
		else
		rd = rd;

浮点条件整数传送
FCIM
	bcnez ca, 1f
	move rd, rj
	1:
		if (ca = 0)
		rd = rj;
		else
		rd = rd;

	bceqz ca, 1f
	move rd, rj
	1:
		if (ca != 0)
		rd = rj;
		else
		rd = rd;

整数条件浮点传送
ICFM
	bnez rk, 1f
	fmov.{s/d} fd, fj
	1:
		if (rk = 0)
		fd = fj;
		else
		fd = fd;

	beqz rk, 1f
	fmov.{s/d} fd, fj
	1:
		if (rk != 0)
		fd = fj;
		else
		fd = fd;

浮点条件浮点传送
FCFM
	fsel fd, fj, fd, ca	if (ca = 0)
		fd = fj;
		else
		fd = fd;

	fsel fd, fd, fj, ca	if (ca != 0)
		fd = fj;
		else
		fd = fd;

3.3.3.2 条件传送机制优化实现

除了浮点条件浮点传送指令 fsel 外，LoongArch 指令集中并没有其他三种类

型的条件传送指令。LoongArch 总共具有 2565 条原生指令，进行猜想：是否存在利用现有的 LoongArch 指令拼凑出其他三种类型的条件传送指令的可能性。

- 整数条件整数传送

经过仔细研读 LoongArch 指令集手册，发现有 maskeqz 和 masknez 指令，它们的具体含义及用法见表 3.25。

表 3.25 maskeqz 和 masknez 指令

Table 3.25 maskeqz and masknez instructions

maskeqz rd, rj, rk
// maskeqz 执行时，如果通用寄存器 rk 的值等于 0 则将通用寄存器 rd 置为全 0，否则将其赋值为 rj 寄存器的值。
masknez rd, rj, rk
// masknez 执行时，如果通用寄存器 rk 的值不等于 0 则将通用寄存器 rd 置为全 0，否则将其赋值为 rj 寄存器的值。

将 maskeqz、masknez 以及 or 三条指令拼凑起来即可实现 LoongArch 指令集架构不带分支的整数条件整数传送指令，见表 3.26。

表 3.26 整数条件整数传送优化

Table 3.26 Integer Condition Integer Move optimization

指令	含义

maskeqz rd, rd, rk
masknez rk, rj, rk
or rd, rd, rk
	if (rk = 0)
	  rd = rj;
	else
	  rd = rd;

masknez rd, rd, rk
maskeqz rk, rj, rk
or rd, rd, rk
	if (rk != 0)
	  rd = rj;
	else
	  rd = rd;

- 浮点条件整数传送

LoongArch 基础指令集中有浮点寄存器与通用寄存器转换指令 movgr2fr 和

movfr2gr，具体用法和含义见表 3.27。

表 3.27 movgr2fr 和 movfr2gr 指令

Table 3.27 movgr2fr and movfr2gr instructions

movgr2fr.w fd, rj
// movgr2fr.w 将通用寄存器 rj 的低 32 位值写入浮点寄存器 fd 的低 32 位中。若浮点寄存器位宽为 64 位，则 fd 的高 32 位值不确定。
movgr2fr.d fd, rj
// movgr2fr.d 将通用寄存器 rj 的 64 位值写入浮点寄存器 fd。

movfr2gr.s rd, fj
// movfr2gr.s 将浮点寄存器 fj 的低 32 位值符号扩展后写入通用寄存器 rd。
movfr2gr.d rd, fj
// movfr2gr.d 将浮点寄存器 fj 的 64 位值写入通用寄存器 rd。

利用浮点条件浮点传送指令 fsel，浮点寄存器与通用寄存器转换指令 movgr2fr 和 movfr2gr，即可拼凑实现 LoongArch 指令集架构不带分支的浮点条件整数传送指令，见表 3.28。

表 3.28 浮点条件整数传送优化

Table 3.28 Floating Condition Integer Move optimization

指令	含义

movgr2fr.d ft1, rd
movgr2fr.d ft2, rj
fsel ft1, ft2, ft1, ca
movfr2gr.d rd, ft1
	if (ca = 0)
	  rd = rj;
	else
	  rd = rd;

movgr2fr.d ft1, rd
movgr2fr.d ft2, rj
fsel ft1, ft1, ft2, ca
movfr2gr.d rd, ft1
	if (ca != 0)
	  rd = rj;
	else
	  rd = rd;

- 整数条件浮点传送

LoongArch 基础指令集中有浮点比较指令 fcmp.cond.{s/d}，这条指令的 cond 有 22 种，如 ceq(相等)、caf(否)、clt(小于)、cne(不等)等情况。接下来仅介

绍本节中将用到的指令 fcmp.ceq.{s/d}，具体用法和含义见表 3.29。

表 3.29 fcmp.ceq 指令

Table 3.29 fcmp.ceq instruction

fcmp.ceq.{s/d} ca, fj, fk // 比较{单精度数/双精度数}fj 和 fk 是否相等，若相等，条件标志寄存器 ca=1,否则 ca=0。

表 3.30 整数条件浮点传送优化

Table 3.30 Integer Condition Floating Move optimization

指令	含义

movgr2fr.d ft1, r0
movgr2fr.d ft2, rk
fcmp.ceq.d ca, ft1, ft2
fsel fd, fd, fj, ca
	if (rk = 0)
	  fd = fj;
	else
	  fd = fd;

movgr2fr.d ft1, r0
movgr2fr.d ft2, rk
fcmp.ceq.d ca, ft1, ft2
fsel fd, fj, fd, ca
	if (rk != 0)
	  fd = fj;
	else
	  fd = fd;

利用浮点条件浮点传送指令 fsel，通用寄存器与浮点寄存器转换指令 movgr2fr.d，浮点比较指令 fcmp.ceq.d，即可拼凑实现 LoongArch 指令集架构不带分支的整数条件浮点传送指令，见表 3.30。

- 小结

表 3.31 列出了 LoongArch 优化前后条件传送机制对照表。LoongArch 优化后，四种类型的条件传送均不再包含分支指令。

3.3.4 小结

将 Java 虚拟机移植到 LoongArch 指令集架构之后，通过测试发现 LoongArch 指令集架构的 Java 虚拟机在 scimark.monte_carlo 测试程序上与 MIPS 有 30%以上的性能差距。利用 perf 工具和使用 Java 虚拟机参数打印 C2 编译器生成的汇编指令，发现影响性能差距的原因在于 LoongArch 指令集架构缺少条件传送指令。为了弥补这一性能差距，通过充分利用 LoongArch 指令集

架构的 2565 条原始指令拼凑出条件传送指令，使得 scimark.monte_carlo 测试程序得分提升 71%，具体的性能效果见 4.4 节。

表 3.31 条件传送机制

Table 3.31 Conditional Move mechanism

类型	LoongArch 优化前	LoongArch 优化后
整数条件 整数传送	bnez rk, 1f move rd, rj 1:	maskeqz rd, rd, rk masknez rk, rj, rk or rd, rd, rk
	beqz rk, 1f move rd, rj 1:	masknez rd, rd, rk maskeqz rk, rj, rk or rd, rd, rk
浮点条件 整数传送	bcnez ca, 1f move rd, rj 1:	movgr2fr.d ft1, rd movgr2fr.d ft2, rj fsel ft1, ft2, ft1, ca movfr2gr.d rd, ft1
	bceqz ca, 1f move rd, rj 1:	movgr2fr.d ft1, rd movgr2fr.d ft2, rj fsel ft1, ft1, ft2, ca movfr2gr.d rd, ft1
整数条件 浮点传送	bnez rk, 1f fmov.{s/d} fd, fj 1:	movgr2fr.d ft1, r0 movgr2fr.d ft2, rk fcmp.ceq.d ca, ft1, ft2 fsel fd, fd, fj, ca
	beqz rk, 1f fmov.{s/d} fd, fj 1:	movgr2fr.d ft1, r0 movgr2fr.d ft2, rk fcmp.ceq.d ca, ft1, ft2 fsel fd, fj, fd, ca
浮点条件 浮点传送	fsel fd, fj, fd, ca	fsel fd, fj, fd, ca
	fsel fd, fd, fj, ca	fsel fd, fd, fj, ca

第4章 龙芯 LoongArch64 位 Java 虚拟机性能评测

本章首先对测试平台和工具进行了简要的介绍，接着对第 3 章的 3 个优化进行了性能评测，最后将 LoongArch 64 位 Java 虚拟机与 MIPS 进行了性能对比。

4.1 测试平台和工具介绍

LoongArch 指令集架构的平台为龙芯 3A5000，MIPS 指令集架构的平台为龙芯 3A4000，它们的主要配置见表 4.1。3A5000 机器默认主频较 3A4000 高，增加一组对比实验，将 3A5000 降频至 1.8GHz 运行 Java 虚拟机测试程序。

表 4.1 测试平台主要配置

Table 4.1 Main configuration of test platform

	龙芯 3A5000	龙芯 3A4000
处理器核架构	LoongArch	MIPS
处理器核数	4	4
工艺	12nm	28nm
主频	2.5GHz	1.8GHz
内存	16GB	16GB
操作系统	Loongnix Desktop 系统	Loongnix 1.0 系统

本文采用的测试工具是 SPECjvm2008[21]。该工具由标准性能评估机构（Standard Performance Evaluation Corporation）提供，是一个用于测量 Java 运行时环境性能的基准测试程序集。该测试工具总共包含 10 类 38 个测试程序，测试结果的单位为 ops/m（每分钟操作数），数值越大性能越好，详细信息见下：

- Startup

  Startup 类程序只运行一次，用来测量 Java 虚拟机的启动性能。

- Compiler

  在规定时间内，多线程迭代测试普通 Java 编译。

- Compress

  在规定时间内，多线程迭代测试压缩。

- Crypto

  Crypto 类程序在规定时间内，多线程迭代测试加解密算法，如 AES/DES、RSA、MD5 等。

- Derby

  在规定时间内，多线程迭代测试数据库相关逻辑。

- Mpegaudio

  在规定时间内，多线程迭代测试 mpeg 音频解码。

- Scimark

  Scimark 类程序又可分为小数据负载和大数据负载两种。小数据负载趋向于测试 Cache 级别的 Java 虚拟机代码优化和计算性能；大数据负载趋向于测试内存级别的代码优化和计算性能。

- Serial

  在规定时间内，多线程迭代测试序列化和反序列化。

- Sunflow

  在规定时间内，多线程迭代测试图片渲染。

- Xml

  在规定时间内，多线程迭代测试 xml 转换和验证。

4.2 解释器翻译性能优化效果

本小节首先测试了 LoongArch 64 位 Java 虚拟机解释器静态生成数据，然后使用 SPECjvm2008 Startup 类程序集测试了 Java 虚拟机的启动性能。

4.2.1 静态数据对比

表 4.2 展示了 LoongArch 优化前后的解释器翻译性能对比， 表 4.3 展示了 LoongArch 优化后与 MIPS 的解释器翻译性能对比。经过立即数加载优化和指令分派优化后，LoongArch 优化后较优化前解释器翻译性能提高 18%左右，优化后的 LoongArch 64 位 Java 虚拟机的解释器翻译性能较 MIPS 高 20%。

表 4.2 优化前后 LoongArch 解释器翻译性能对比

Table 4.2 Comparison of LoongArch interpreter translation performance before and after optimization

评价指标	LoongArch 优化前	LoongArch 优化后	降低率
汇编指令条数	17476 条	14376 条	17.74%
code size	158KB	128KB	18.99%

表 4.3 LoongArch 优化后与 MIPS 解释器翻译性能对比

Table 4.3 Comparison of interpreter translation performance between optimized LoongArch and MIPS

评价指标	MIPS	LoongArch 优化后	降低率
汇编指令条数	17984 条	14376 条	20.06%
code size	163KB	128KB	21.47%

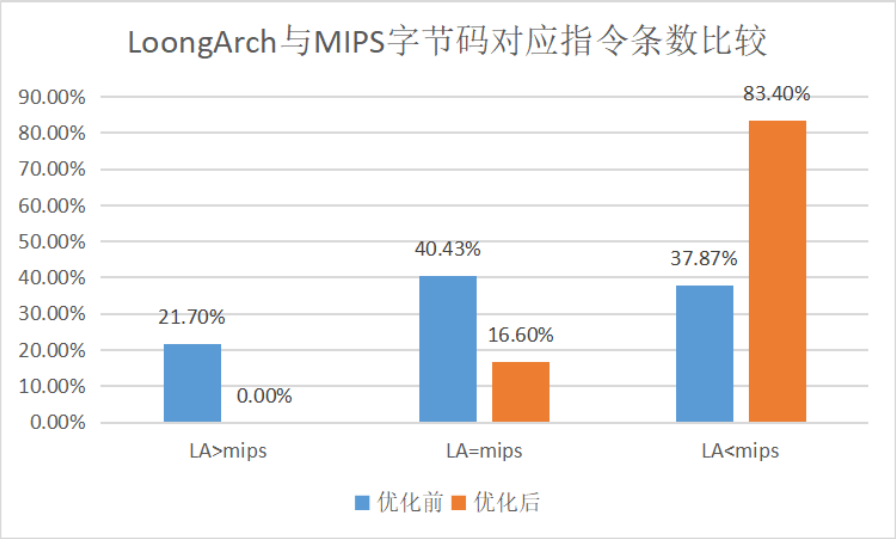

图 4.1 优化前后 LoongArch 与 MIPS 字节码对应指令条数比较

Figure 4.1 Comparison of the number of instructions corresponding to LoongArch and MIPS bytecode before and after optimization

图 4.1 展示了优化前后 LoongArch 与 MIPS 字节码对应指令条数比较，优

化前有 21.7%的字节码 LoongArch 对应的指令数大于 MIPS，优化后 LoongArch 没有字节码对应的指令数大于 MIPS；优化前有 37.9%的字节码 LoongArch 对应的指令数小于 MIPS，优化后这一比例提升至 83.4%。

4.2.2 启动性能

解释器的翻译性能和 Java 虚拟机的启动性能是息息相关的。SPECjvm2008 测试程序集中的 Startup 类程序能够很好的测量 Java 虚拟机的启动性能[21]。在纯解释器模式下，LoongArch Java 虚拟机优化前后的启动性能见表 4.4。经过立即数加载优化和指令分派优化后，LoongArch Java 虚拟机启动性能平均增长 8.94%，其中 startup.sunflow 程序提升比高达 40.24%。

表 4.4 LoongArch Java 虚拟机启动性能

Table 4.4 LoongArch Java virtual machine startup performance

Benchmark	优化前（ops/m）	优化后（ops/m）	增长率
startup.helloworld	139.86	153.7167	9.91%
startup.compiler.compiler	4.36	4.483333	2.83%
startup.compiler.sunflow	4.95	5.34	7.88%
startup.compress	23.26	24.81	6.66%
startup.crypto.aes	6.98	7.986667	14.42%
startup.crypto.rsa	43.6	44.54667	2.17%
startup.crypto.signverify	29.59	30.22667	2.15%
startup.mpegaudio	11.98	12.8	6.84%
startup.scimark.fft	38.17	40.65	6.50%
startup.scimark.lu	35.36	37.52333	6.12%
startup.scimark.monte_carlo	11.84	12.25333	3.49%
startup.scimark.sor	16.14	16.28667	0.91%
startup.scimark.sparse	17.08	21.58667	26.39%
startup.serial	9.07	9.78	7.83%
startup.sunflow	14.82	20.78333	40.24%
startup.xml.transform	1.33	1.373333	3.26%
startup.xml.validation	10.96	11.44333	4.41%

4.3 跳转机制优化性能评测

本小节首先对比了跳转机制优化前后解释器的翻译性能，然后使用 SPECjvm2008 测试程序对比了优化前后的性能效果。

- 解释器翻译性能

Javac 编译器将高级程序语言 Java 语言翻译成中间表示——字节码，解释器逐一地把字节码翻译成平台相关的汇编指令。优化前、后解释器翻译性能对比见表 4.5。Java 语言中总共有 235 个字节码，对跳转机制进行优化后，将近 1/3（77/235）的字节码对应的汇编指令条数减少，总的汇编指令条数从 15780 条减少到 14816 条，指令条数减少了 6.11%。跳转优化后解释器生成代码为 131KB，相比优化前减少了 11KB，内存空间利用率提升了 7.75%。表 4.6 列出了 77 个字节码汇编指令条数减少百分比，绝大部分字节码汇编指令减少百分比在 10%左右，其中 athrow 字节码汇编指令减少百分比最多，高达 50%。

表 4.5 优化前、后解释器翻译性能对比

Table 4.5 Comparison of interpreter translation performance before and after optimization

评价指标	优化前	优化后	提升(百分比)
汇编指令条数	15780 条	14816 条	6.11%
code size	142KB	131KB	7.75%

表 4.6 字节码对应汇编指令条数减少百分比

Table 4.6 Percentage reduction of assembly instructions corresponding to bytecode

指令条数减少百分比	字节码个数
(0, 10%]	31
(10%, 20%]	39
(20%, 30%]	5
(30%, 40%]	1
(40%, 50%]	1
合计	77

- SPECjvm2008 性能测试结果

表 4.7 列出了跳转机制优化前后 SPECjvm2008 性能数据以及性能提升百分比。由表可知，优化以后，性能最高有 4.24%的提升，平均性能提升幅度为 1.59%。

表 4.7 跳转机制优化前、后 SPECjvm2008 测试数据对比表

Table 4.7 Comparison of SPECjvm2008 test data before and after jump mechanism optimization

Benchmark	优化前三次平均值 (ops/m)	优化后三次平均值 (ops/m)	性能提升 (百分比)
compiler.compiler	261.01	267.78	2.59%
compiler.sunflow	94.13	96.91	2.95%
compress	124.87	126.36	1.19%
crypto.aes	32.25	32.40	0.45%
crypto.rsa	473.78	476.37	0.55%
crypto.signverify	238.09	239.15	0.45%
derby	152.04	154.42	1.57%
scimark.fft.large	19.95	20.52	2.84%
scimark.lu.large	10.27	10.32	0.49%
scimark.sor.large	16.91	16.91	0.00%
scimark.sparse.large	26.17	26.65	1.85%
scimark.monte_carlo	74.69	76.40	2.29%
scimark.fft.small	257.08	259.03	0.76%
scimark.lu.small	228.65	228.76	0.05%
scimark.sor.small	73.58	73.33	-0.34%
scimark.sparse.small	95.62	99.25	3.79%
serial	60.84	63.41	4.24%
xml.transform	165.66	167.20	0.93%
xml.validation	277.66	287.70	3.62%

跳转机制优化大幅度降低了各种跳转情况所需要的指令条数，然而平均性能提升幅度仅有 1.59%，并不明显。造成这个现象的原因在于：Java 程序中大

部分指令都是顺序指令，跳转指令并不是 Java 程序的热点所在。跳转机制优化更大地意义在于简化了 Java 虚拟机中跳转机制的实现，patch 时只需要简单地考虑两种情况，对 Java 虚拟机的开发人员提供了很大地便利。

4.4 条件传送机制优化性能评测

本小节首先使用自编写的嵌入式汇编小程序对比了条件传送指令和分支指令的性能差异，然后对比测试了 scimark.monte_carlo 程序性能。

4.4.1 精简程序性能

图 4.3 展示了自编写的嵌入式汇编小程序，该程序将 LoongArch 优化前和优化后的整数条件整数传送指令集模拟实际应用场景运行 3 亿次，比较优化前后两者的差异。其他三种类型同理，就不在此一一赘述了。优化前后四种条件传送类型的运行时间和分支误预测率分别见图 4.2 和表 4.8，其中浮点条件浮点传送类型不需要进行优化，因此优化前、后数据一致。通过图 4.2 可以看到，优化后的条件传送机制指令集运行时间大大缩短，平均仅需要优化前 1/3 的时间；通过表 4.8 可以看到，采用分支指令实现的条件传送机制平均分支误预测率高达 20%，而优化后的浮点传送机制分支误预测率为 0.00%。

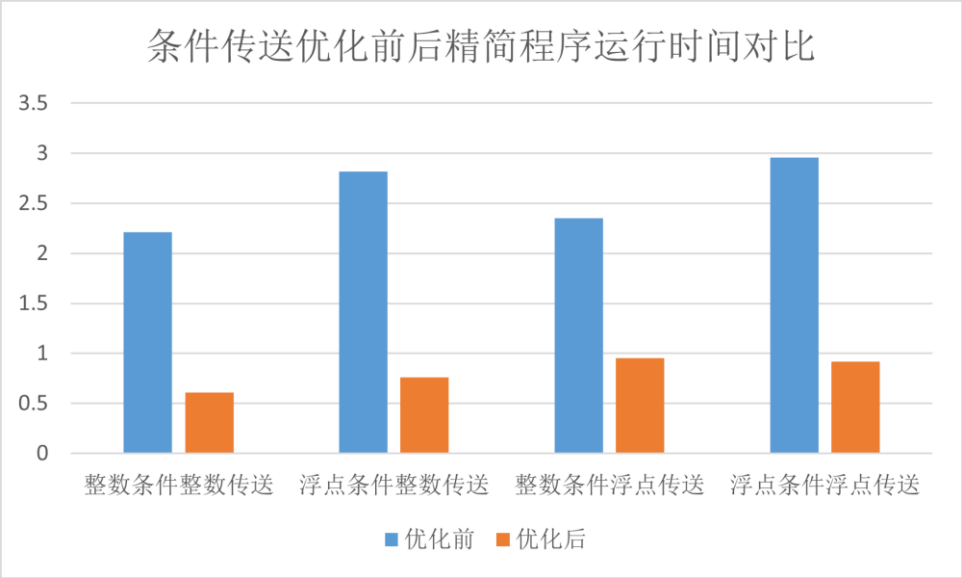

图 4.2 条件传送优化前后精简程序运行时间对比

Figure 4.2 Comparison of running time of streamlined program before and after conditional move optimization

表 4.8 条件传送优化前后精简程序分支误预测率对比

Table 4.8 Comparison of branch-miss rate before and after conditional move optimization

类型	优化前	优化后
整数条件整数传送	20.65%	0.00%
浮点条件整数传送	14.55%	0.00%
整数条件浮点传送	21.75%	0.00%
浮点条件浮点传送	0.00%	0.00%

```
for (k = 0; k < 10000; k++) {
    for (i = 0; i < 32768; i++) {
        int v, t, r = rands[i];
        asm volatile (
            "move    %[v], $zero      \n\t"
            "slti    %[t], %[r], 128  \n\t"
#ifdef CMOV
            "masknez %[v], %[v], %[t] \n\t"
            "maskeqz %[t], %[r], %[t] \n\t"
            "or      %[v], %[v], %[t] \n\t"
#else
            "beqz    %[t], 1f         \n\t"
            "move    %[v], %[r]       \n\t"
            "1:                       \n\t"
#endif
            :[v]"=&r"(v), [t]"=&r"(t)
            :[r]"r"(r)
            :
        );
```

图 4.3 条件传送机制精简测试

Figure 4.3 Conditional move mechanism simplification test

4.4.2 scimark.monte_carlo 程序性能

表 4.9 列出了条件传送机制优化前后 scimark.monte_carlo 程序性能，可以看到 scimark.monte_carlo 的分数从 78.22 提升至 133.76 分，增长率高达 71%；

分支误预测率从 2.99%下降至 0.01%。

表 4.10 列出了 scimark.monte_carlo 程序 MIPS 与 LoongArch 性能比较，LoongArch 优化前，性能较 MIPS 低 32.17%；优化后，LoongArch 性能较 MIPS 高 26.06%。优化前分支误预测率较 MIPS 高达 150 倍，优化后分支误预测率仅为 MIPS 的 1/2。

表 4.9 条件传送机制优化前后 scimark.monte_carlo 程序性能比较

Table 4.9 Performance comparison of scimark.monte_carlo before and after Conditional Move mechanism optimization

	优化前	优化后
分数 （ops/m）	78.22	133.76
branch-miss rate	2.99%	0.01%

表 4.10 scimark.monte_carlo 程序 MIPS 与 LoongArch 性能比较

Table 4.10 Performance comparison of scimark.monte_carlo between MIPS and LoongArch

	MIPS	LoongArch
分数（ops/m）	106.11	133.76
branch-miss rate	0.02%	0.01%

4.5 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比与分析

结合上述所有优化后，采用 SPECjvm2008 对 1.8GHz 主频 MIPS 架构处理器 Java 虚拟机、1.8GHz 主频 LoongArch 架构处理器 Java 虚拟机、2.5GHz 主频 LoongArch 架构处理器 Java 虚拟机进行了性能测试，具体的测试结果见表 4.11。

由表中数据可见，在 1.8GHz 的主频下，LoongArch 架构 Java 虚拟机比 MIPS 架构的 Java 虚拟机平均性能提升 21.87%，性能最高提升 42.48%。

在 2.5GHz 的主频下，LoongArch 架构的 Java 虚拟机比 1.8GHz 主频 MIPS 架构的 Java 虚拟机平均性能提升 64.70%，性能最高提升 97.06%。

表 4.11 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比

Table 4.11 Performance comparison between LoongArch and MIPS 64 bit Java virtual machine

Benchmark	MIPS 1.8GHz （ops/m）	LoongArch 1.8GHz （ops/m）	增长率	LoongArch 2.5GHz （ops/m）	增长率
compiler.compiler	210.09	289.01	37.56%	388.86	85.09%
compiler.sunflow	73.69	104.99	42.48%	111.4	51.17%
compress	120.21	133.07	10.70%	185.3	54.15%
crypto.aes	34.99	38.17	9.10%	52.96	51.36%
crypto.rsa	463.21	590.59	27.50%	810.69	75.02%
crypto.signverify	211.83	254.23	20.01%	351.59	65.98%
derby	116.01	159.33	37.34%	204.12	75.95%
mpegaudio	66.76	83.47	25.03%	115.68	73.28%
scimark.fft.large	26.94	24.07	-10.64%	22.74	-15.59%
scimark.lu.large	7.13	9.99	40.16%	11.78	65.22%
scimark.sor.large	14.04	16.67	18.76%	23.1	64.53%
scimark.sparse.large	19.35	25.05	29.44%	31.46	62.58%
scimark.fft.small	250.97	280.81	11.89%	382.15	52.27%
scimark.lu.small	221.56	238.84	7.80%	342.29	54.49%
scimark.sor.small	61.21	73.01	19.28%	101.27	65.45%
scimark.sparse.small	78.16	109.99	40.72%	154.02	97.06%
scimark.monte_carlo	124.8	151.94	21.75%	215.06	72.32%
serial	53.55	66.29	23.78%	91.2	70.31%
sunflow	31.35	38.54	22.93%	52.87	68.64%
xml.transform	141.41	183.27	29.60%	250.34	77.03%
xml.validation	237.11	325.67	37.35%	435.49	83.67%

图 4.4 展示了三者的性能对比效果图，其中横坐标为 SPECjvm2008 的各个测试程序，纵坐标是以 1.8GHz 主频的 MIPS 架构 Java 虚拟机的得分为基准进

行了归一化处理，数值越大代表性能越好。

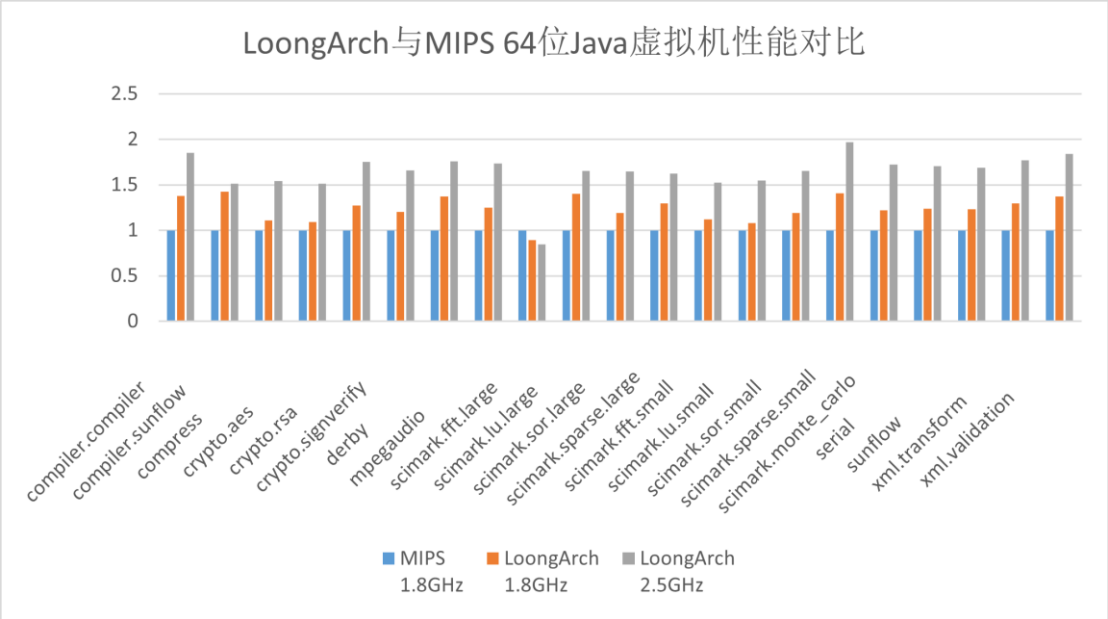

图 4.4 LoongArch 与 MIPS 64 位 Java 虚拟机性能对比

Figure 4.4 Performance comparison between LoongArch and MIPS 64 bit Java virtual machine

在 scimark.fft.large 测试程序上，LoongArch 架构的 Java 虚拟机比 MIPS 架构的 Java 虚拟机性能有一定的下降。通过对测试程序和虚拟机行为进行分析，发现 LoongArch 架构的 Java 虚拟机 IPC 很低，同时 L1-dcache-miss-rate 很高。通过关闭 3A5000 机器的 SCache LRU 替换策略，LoongArch 架构的 Java 虚拟机 scimark.fft.large 程序跑分增长 23.73%。因此认为影响该程序性能差异的原因不在 Java 虚拟机层面，而在于硬件层面。

第5章 总结与展望

5.1 本文工作总结

本文工作主要集中于以下三个方面：

第一，完成龙芯 LoongArch 64 位 Java 虚拟机移植。以 OpenJDK8 为基础，将 Java 虚拟机中和平台相关的部分，如汇编器、解释器和编译器等移植到 LoongArch 指令集架构处理器上。

第二，完成龙芯 LoongArch 64 位 Java 虚拟机优化。基于性能测试结果和 LoongArch 指令集架构特点，首先通过优化立即数加载函数和使用移位运算指令和双寄存器访存指令优化指令分派过程，对解释器的翻译性能进行了优化；接着通过使用相对 PC 运算指令和调整 CodeCache 虚拟内存映射空间对 LoongArch 64 位 Java 虚拟机的跳转机制进行了优化；最后通过使用多条 LoongArch 基础指令拼凑出条件传送指令对 LoongArch 64 位 Java 虚拟机的条件传送机制进行了优化。

第三，进行龙芯 LoongArch 64 位 Java 虚拟机性能评测。首先对上述三个优化的优化效果进行了性能评测。经过解释器的翻译性能优化后，Java 虚拟机字节码对应汇编指令的生成条数相比优化之前下降 18%，同时 Java 虚拟机启动性能平均增长 8.94%；经过跳转机制优化后，有将近 1/3 的字节码对应的汇编指令生成条数减少，同时 SPECjvm2008 平均性能提升幅度为 1.59%，最高有 4.24%的提升；经过条件传送机制优化后，LoongArch 64 位 Java 虚拟机在 scimark.monte_carlo 程序上性能增长高达 71%，同时该程序的分支误预测率从 2.99%降低至 0.01%。接着将 LoongArch 64 位 Java 虚拟机的性能和 MIPS 进行了对比，结果表明：LoongArch 64 位 Java 虚拟机在同主频 LoongArch 架构处理器相比于 MIPS 架构处理器的 Java 虚拟机平均性能提升 21.87%，性能最高提升 42.48%。运行在 2.5GHz 主频的 LoongArch 架构处理器的 Java 虚拟机比运行在 1.8GHz 主频 MIPS 架构处理器的 Java 虚拟机平均性能提升 64.70%，性能最高提升 97.06%。

本文的研究成果已经集中到龙芯 OpenJDK8 版本库中，并投入实际生产。

5.2 龙芯 LoongArch64 位指令集的改进方向

在本文的工作和实验过程中，发现龙芯 LoongArch 64 位指令集在 Java 虚拟机上有以下两个改进方向：

第一，LoongArch 64 位指令集需要增加条件传送指令。从 3.3 节和 0 节中可知，进行条件传送机制优化之前，LoongArch 64 位 Java 虚拟机在 scimark.monte_carlo 程序较 MIPS 64 位 Java 虚拟机性能低 30%。目前利用多条 LoongArch 基础指令拼凑出条件传送指令，使 LoongArch 64 位 Java 虚拟机在 scimark.monte_carlo 程序性能增长高达 71%，同时分支误预测率也从 2.99%降低至 0.01%。若 LoongArch 指令集中有单条条件传送指令，显然会比当前用多条指令拼凑出的性能效果更好。

第二，LoongArch 64 位指令集需要增加加法进位指令。在与 ARM 机器的性能比较中，发现 LoongArch 64 位 Java 虚拟机在 crypto.rsa 测试程序中性能较差，换算成同主频性能只有 ARM 机器的 55%。通过分析，crypto.rsa 测试程序执行过程主要在执行蒙哥马利算法，蒙哥马利算法中需要进行多次带进位加法。LoongArch 64 位指令集没有带进位加法指令，目前是利用 sltu 指令将进位保存在寄存器中实现的。ARM[32]和 X86[33]指令集架构中均有带进位加法指令。将 ARM 架构 Java 虚拟机中的蒙哥马利算法改成类似于 LoongArch 的实现方式，ARM 机器在 crypto.rsa 测试程序的得分和 LoongArch 接近。因此 LoongArch 64 位指令集中若有加法进位指令，LoongArch 64 位 Java 虚拟机的性能效果会有一定的提升。对于需要频繁用到加法进位指令的加密解密类程序效果将会更显著。

5.3 下一步工作和研究方向

下一步工作和研究方向将从以下三个方面展开：

第一，同主频 LoongArch 64 位 Java 虚拟机比 MIPS 64 位 Java 虚拟机平均性能提高了 21.87%，但是与 X86、ARM 架构的 Java 虚拟机在 serial，scimark.fft.large，crypto.signverify 等测试程序上还有一定的性能差距。接下来争取分析出这些测试程序在 LoongArch 64 位 Java 虚拟机上的性能瓶颈，缩小与 X86、ARM 架构的 Java 虚拟机的性能差距，进一步提高 LoongArch 64 位 Java 虚拟机性能。

第二，由于时间有限，目前仅将 LoongArch 基础指令集移植到 Java 虚拟机，而 128 位向量扩展指令和高级向量扩展指令还未移植到 Java 虚拟机。下一步将向量指令集移植到 Java 虚拟机，这将在一定程度上提升 LoongArch 64 位 Java 虚拟机的性能。

第三，本文采用的测试工具为标准性能测试工具集 SPECjvm2008，接下来还可以选取更多的 JVM 基准测试程序集进行测试和分析。一方面能够使得测试数据可以详细可靠，另一方面也可以从多维度对测试结果进行分析，有利于 LoongArch 64 位虚拟机的持续演进。

参考文献

[1] 胡伟武. 指令系统的自主与兼容. CCF 全国计算机体系结构学术年会, 2020.

[2] 龙芯架构参考手册 卷一：基础架构及虚拟化扩展 V0.10。龙芯中科技术有限公司，2020.6。

[3] 龙芯指令集手册 （卷Ⅱ-a）——自定义通用扩展指令（上册） v1.00。龙芯中科技术有限公司，2012.3。

[4] 刘超 . 基于龙芯处理器的 Java 运行环境的移植与优化 [D]. 北京：中科院计算所，2006.

[5] 陈杰 . HotSpot 虚拟机在"龙芯"Linux 系统上的移植与性能优化[D]. 中国科学院计算技术研究所, 2007.

[6] 伍鸣 . Java 虚拟机的内存管理及优化[D]. 中国科学院计算技术研究所, 2007.

[7] 杨勇强 . 基于四核龙芯 3 号平台的 HotSpot6 移植与优化 [D]. 北京：中科院计算所， 2012.

[8] 蔡嵩松 . 软硬件协同设计的 Java 虚拟机系统实现与性能优化[D]. 中国科学院研究生院 中国科学院大学, 2011.

[9] 武慧 . 龙芯 MIPS 64 位 Java 虚拟机的实现与优化 [D]. 北京：中科院计算所，2013.

[10] 王一光 . MIPS 64 平台下 JVM 性能优化研究与实现 [D]. 北京：中科院计算所，2015.

[11] 敖琪. 高可靠高性能的 Java 虚拟机研究 [D]. 北京：中科院计算所， 2016.

[12] 傅杰 . Java 虚拟机性能优化技术研究 [D]. 北京：中科院计算所， 2016.

[13] 陈昊轩 . 基于 MIPS 平台的 Java 虚拟机性能优化方法研究 [D]. 北京：中科院计算所， 2018.

[14] TIOBE. TIOBE 编程语言排行榜. http://www.tiobe.com/tiobe_index.

[15] Java.了解 Java 技术. https://www.java.com/zh-CN/about/

[16] 周志明. 深入理解 Java 虚拟机：JVM 高级特性与最佳实践. 北京：机械工业出版社, 2014.

[17] https://openjdk.java.net/jeps/

[18] https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-6.html

[19] Sweetman D. See MIPS run[M]. Elsevier, 2010.

[20] Emer J S, Edwards B, Leibholz D L, et al. Implementation of a conditional move instruction in an out-of-order processor: U.S. Patent 6,449,713[P]. 2002-9-10.

[21] https://www.spec.org/jvm2008/

[22] Kotzmann T, Wimmer C, Mössenböck H, et al. Design of the Java HotSpot™ client compiler

for Java 6[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2008, 5(1): 1-32.

[23] Paleczny M, Vick C, Click C. The java hotspottm server compiler[C]//Proceedings of the Java Virtual Machine Research and Technology Symposium. 2001, 1: 1-12.

[24] Würthinger T, Wimmer C, Mössenböck H. Visualization of program dependence graphs[C]//International Conference on Compiler Construction. Springer, Berlin, Heidelberg, 2008: 193-196.

[25] Ertl M A, Gregg D. The behavior of efficient virtual machine interpreters on modern architectures[C]//European Conference on Parallel Processing. Springer, Berlin, Heidelberg, 2001: 403-413.

[26] Venkatachalam V , Engineer S S . A Tutorial on Adding New Instructions to the Oracle® Java HotSpot ™ Virtual Machine.

[27] Fraser C W, Hanson D R, Proebsting T A. Engineering a simple, efficient code-generator generator[J]. ACM Letters on Programming Languages and Systems (LOPLAS), 1992, 1(3): 213-226.

[28] Tim Lindholm, Frank Yellin, Gilad Bracha, Alex BuckleyF . The Java Virtual Machine Specification, Java SE 9 Edition. 2017.

[29] Aycock J. A brief history of just-in-time[J]. ACM Computing Surveys (CSUR), 2003, 35(2): 97-113.

[30] Rossi M, Sivalingam K. A survey of instruction dispatch techniques for byte-code interpreters[C]//SEMINAR ON MOBILE CODE, NUMBER TKO-C-79, LABORATORY OF INFORMATION PROCESSING SCIENCE, HELSINKI UNIVERSITY OF TECHNOLOGY, 1995. BIBLIOGRAPHY 92 [77. 1996.

[31] MIPS Ⓡ Architecture For Programmers[J]. Volume II-A: The MIPS32Ⓡ Instruction Set.

[32] ARM A R M. Architecture Reference Manual-ARMv8, for ARMv8-A architecture profile[J]. ARM Limited, Dec, 2017.

[33] Intel I. and IA-32 architectures software developer’s manual[J]. Volume 3A: System Programming Guide, Part, 64, 1(64): 64.

[34] Briggs P, Cooper K D, Simpson L T. Value numbering[J]. Software: Practice and Experience, 1997, 27(6): 701-724.

[35] Knoop J, Rüthing O, Steffen B. Partial dead code elimination[J]. ACM SIGPLAN Notices, 1994, 29(6): 147-158.

[36] Mccandless J, Gregg D. Compiler techniques to improve dynamic branch prediction for indirect jump and call instructions[J]. ACM Transactions on Architecture and Code Optimization (TACO), 2012, 8(4): 1-20.

[37] Detlefs D, Agesen O. Inlining of virtual methods[C]//European Conference on Object-Oriented Programming. Springer, Berlin, Heidelberg, 1999: 258-277.

[38] Cramer T, Friedman R, Miller T, et al. Compiling Java just in time[J]. Ieee micro, 1997,

17(3): 36-43.

[39] JVM Team. The java HotSpot virtual machine[J]. Technical White Paper, Sun Microsystems, Tech. Rep, 2006.

致 谢

研究生三年的时光转瞬即逝，这三年我成长了很多，也收获了很多。在此，谨向所有关心和帮助我的人表示由衷的感谢。

首先，感谢我的组长敖琪师兄。无论是工作和生活上，师兄都给我提供了很多帮助。师兄认真严谨的工作态度，敏锐的洞察能力，优秀的管理能力，都让我敬佩不已。如果没有师兄的耐心指导，我的工程实践和学术论文进展不会这么顺利。

感谢我的导师章隆兵老师在科研与学业上给我提供了无私的指导，同时章老师对工作的认真负责，对科研的耐心热枕，对学生的和蔼可亲都深深地印入我的脑海。感谢王剑老师在研究生期间给予我科研和生活上的指导与帮助。感谢肖俊华老师从大四到研二不厌其烦地督促我们读论文，给我的科研和工程打下了一个良好的基础。

感谢 Java 组的王锐。论文工程中大大小小的优化和工程实现，如果没有你的帮助，我的毕设进展不会如此顺利。同时你分析问题、解决问题的能力值得我好好学习。

感谢 Java 组的王雪、王浩敏、孙国云、孙亚奇、蒋新尚、潘学峰、翟翔等所有 Java 组的小伙伴。无论是大的问题、小的问题，甚至是弱智的问题，你们都不厌其烦一一帮我解答。

感谢刘先喆、牛根、邹旭、黄科乐、谭弘泽、王佃涛和谢本壹等 2018 届微处理器中心的所有小伙伴。大家互相玩耍、互相帮助、共同奋斗、一起成长，度过了愉快又温暖的三年。

感谢邢世远同学。谢谢你的包容、理解和陪伴，研究生阶段能和你一起度过，我感到非常幸运。

感谢我最亲爱的家人们，你们永远都无条件地支持我、鼓励我、关心我和爱护我。因为有你们，无论遇到什么事情，我都可以勇敢地去面对。

黄佳

2021 年 6 月

作者简历及攻读学位期间发表的学术论文与研究成果

作者简历：

2014 年 9 月——2018 年 6 月，在北京交通大学计算机与信息技术学院获得学士学位。

2018 年 9 月——2021 年 6 月，在中国科学院计算技术研究所攻读硕士学位。

获奖情况：

2019 年 6 月　中国科学院大学　三好学生

2020 年 6 月　中国科学院大学　三好学生

工作经历：

已发表（或正式接受）的学术论文：

参加的研究项目及获奖情况：